

BUSINESS ANALYTICS PROJECT

Retail Sales Analysis and Business Decision Making Using Business Analytics

Student Name: Priti Yadav

Roll Number: T054

Class: B.Sc. Information Technology

Division: A

Subject: Applied Business Analytics

Academic Year: 2026–2027

College Name: SHETH L.U.J & SIR MV COLLAGE

Department Name: IT

Faculty Name: Ms. Chetana Tanawade

TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. Objectives
4. Business Scenario / Domain Description
5. Dataset Description
6. Data Collection
7. Data Preprocessing
8. Exploratory Data Analysis
9. Descriptive Statistical Analysis
10. Distribution Analysis
11. Correlation Analysis
12. Regression Analysis
13. Business Intelligence Dashboard
14. Probability and Probability Distributions
15. Decision Making Under Uncertainty
16. Sampling and Sampling Distribution
17. Confidence Interval Estimation
18. Bootstrapping
19. Hypothesis Testing
20. A/B Testing
21. Time Series Analysis
22. Forecast Accuracy
23. Optimization Modeling
24. Simulation Modeling
25. ANOVA
26. Experimental Design
27. Factorial Experiment
28. Business Insights and Findings
29. Business Recommendations
30. Conclusion
31. Appendix

1. INTRODUCTION

Business Analytics is the process of collecting, analyzing and interpreting data to understand business performance and support better decision-making. It helps businesses identify patterns, trends and relationships in their data.

Data is important for businesses because it provides useful information about sales, products, customers, suppliers and business operations. By analyzing data, a business can understand its performance, identify problems and make decisions based on actual information rather than assumptions.

In this project, the selected business area is Retail Sales and Warehouse Management. The selected CSV dataset, Retail and warehouse Sale.csv, contains information such as year, month, supplier, item code, item description, item type, retail sales, retail transfers and warehouse sales.

This business area was selected because retail businesses need to manage sales, products, suppliers and warehouse activities efficiently. Large amounts of sales and inventory-related data can be difficult to understand without proper analysis.

Business Analytics can help this business by analyzing retail sales, retail transfers and warehouse sales, comparing different item types, identifying sales patterns and understanding relationships between business variables. The

analysis can support better sales planning, inventory management, product decisions and overall business decision-making.

2. PROBLEM STATEMENT

The selected retail business faces the challenge of understanding and managing its retail sales and warehouse activities effectively. The business has data related to different suppliers, products, item types, retail sales, retail transfers and warehouse sales, but this data needs to be analyzed to identify useful patterns and trends.

The available dataset Retail and warehouse Sale.csv contains information such as year, month, supplier, item code, item description, item type, retail sales, retail transfers and warehouse sales.

The project aims to analyze this data to understand sales performance, product-wise and item-type-wise sales, warehouse activities, relationships between variables, and sales trends over time. Statistical and Business Analytics techniques will be used to study the data and identify meaningful business patterns.

The analysis will help in making decisions related to sales planning, inventory and warehouse management, product performance, supplier management and future business planning.

3. OBJECTIVES

1. To collect and understand the selected retail sales and warehouse dataset.
2. To perform data preprocessing and exploratory data analysis.
3. To calculate descriptive statistics and study data distributions.
4. To identify relationships between variables using correlation and regression analysis.
5. To analyze probability and apply decision-making techniques under uncertainty.
6. To perform sampling, confidence interval estimation, bootstrapping and hypothesis testing.
7. To forecast future sales and compare forecasting accuracy.
8. To apply optimization, simulation, ANOVA and experimental design techniques.

4. BUSINESS SCENARIO / DOMAIN DESCRIPTION

Business Name

ABC Retail and Warehouse Store

Type of Business

ABC Retail and Warehouse Store is a retail business that sells different types of products to customers and manages stock through warehouse operations.

Products/Services Offered

The business offers a variety of products belonging to different item types. It manages the sale and transfer of products between retail and warehouse operations.

Target Customers

The target customers are individual consumers and regular retail customers who purchase products according to their needs.

Business Operations

The main operations of the business include purchasing products from suppliers, maintaining product information, selling products through retail channels, transferring products, and managing warehouse stock. The business records information such as supplier, item code, item description, item type, retail sales, retail transfers and warehouse sales.

Important Business Problems

The business needs to understand its sales performance, product demand, warehouse activities and sales trends. It may also face challenges in managing inventory, identifying better-performing products and planning future sales. Therefore, Business Analytics can be used to analyze the available data and provide useful insights for sales planning, inventory management, warehouse management and better business decision-making.

5. DATASET DESCRIPTION

The dataset selected for this project is Retail and warehouse Sale.csv. It contains retail and warehouse sales information that can be used to study product performance, supplier activities, retail sales, transfers and warehouse sales.

Dataset Information:

Information	Details
Dataset Name	Retail and warehouse Sale.csv
Source	Selected retail and warehouse sales dataset
Number of Records	30,000 records
Number of Columns	9 columns
Business Domain	Retail Sales and Warehouse Management

Description of Columns:

Column	Description	Data Type	Variable Type
YEAR	Year in which the sales record was recorded	Integer	Numerical
MONTH	Month of the sales record	Integer	Numerical
SUPPLIER	Name/identifier of the product supplier	Object	Categorical
ITEM CODE	Unique code assigned to an item	Object	Categorical
ITEM DESCRIPTION	Description/name of the item	Object	Categorical
ITEM TYPE	Category or type of the item	Object	Categorical
RETAIL SALES	Quantity/value of products sold through retail	Float	Numerical
RETAIL TRANSFERS	Quantity of products transferred through retail operations	Float	Numerical
WAREHOUSE SALES	Quantity/value of products sold through warehouse operations	Float	Numerical

Numerical Variables

The main numerical variables are:

- YEAR
- MONTH
- RETAIL SALES
- RETAIL TRANSFERS
- WAREHOUSE SALES

Categorical Variables

The categorical variables are:

- SUPPLIER
- ITEM CODE
- ITEM DESCRIPTION
- ITEM TYPE

6. DATA COLLECTION

The data used for this project was obtained from a publicly available retail and warehouse sales dataset. The dataset was selected because it contains relevant information required to analyze retail sales and warehouse operations.

Data Source:

Dataset: Retail and wherehouse Sale.csv

Source: Publicly available dataset

Data Collection Method:

The dataset was collected as a pre-existing secondary dataset rather than through a survey. The data was downloaded and imported into Python using the Pandas library for analysis.

Number of Records:

The dataset contains 30,000 records and 9 variables.

Time Period:

The dataset contains year and month information for the sales records. The exact time period will be reported based on the YEAR and MONTH values present in the dataset.

Variables Collected:

The dataset contains the following variables:

- YEAR
- MONTH
- SUPPLIER
- ITEM CODE
- ITEM DESCRIPTION
- ITEM TYPE
- RETAIL SALES
- RETAIL TRANSFERS
- WAREHOUSE SALES

The collected data will be used for data preprocessing, exploratory analysis, statistical analysis, forecasting, optimization, simulation and business decision-making.

7. DATA PREPROCESSING

Data preprocessing was performed before analysis to improve the quality and consistency of the Retail and wherehouse Sale.csv dataset. The following checks were performed.

1. Check Missing Values:

Missing values were checked in all columns using `isnull().sum()`. If any missing values are found, they can be handled by removing the affected records or replacing the values where appropriate.

2. Check Duplicate Records:

Duplicate records were checked using `duplicated().sum()`. Duplicate records, if present, can be removed to avoid counting the same information more than once.

3. Check Data Types:

The data types of all columns were checked using `dtypes`. Numerical columns such as YEAR, MONTH, RETAIL SALES, RETAIL TRANSFERS and WAREHOUSE SALES should contain numerical values, while supplier and item-related columns should be categorical/text data.

4. Check Invalid Values:

The numerical columns were checked for invalid or negative values. Values that are not meaningful for sales or transfer quantities should be reviewed and corrected or removed if necessary.

5. Check Outliers:

Outliers in important numerical variables such as RETAIL SALES, RETAIL TRANSFERS and WAREHOUSE SALES were identified using box plots. Outliers were reviewed before performing statistical analysis.

6. Check Inconsistent Values

Categorical columns such as SUPPLIER, ITEM DESCRIPTION and ITEM TYPE were checked for inconsistent or incorrectly entered values. Similar values with different spelling or formatting were standardized where required.

CODE:

```
import pandas as pd
# Load dataset
df = pd.read_csv("Retail and warehouse Sale.csv")
# 1. Missing values
print("Missing Values:")
print(df.isnull().sum())
# 2. Duplicate records
print("\nDuplicate Records:")
print(df.duplicated().sum())
# 3. Data types
print("\nData Types:")
print(df.dtypes)
# 4. Check invalid negative values
print("\nNegative Values:")
print(df[["RETAIL SALES", "RETAIL TRANSFERS",
          "WAREHOUSE SALES"]].lt(0).sum())
# 5. Check numerical summary for outliers
print("\nNumerical Summary:")
print(df[["RETAIL SALES", "RETAIL TRANSFERS",
          "WAREHOUSE SALES"]].describe())
# 6. Check categorical values
print("\nItem Types:")
print(df["ITEM TYPE"].unique())
print("\nSuppliers:")
print(df["SUPPLIER"].unique())
```

OUTPUT:

Missing Values:		Duplicate Records:	
YEAR	0	0	
MONTH	0	Data Types:	
SUPPLIER	33	YEAR	int64
ITEM CODE	0	MONTH	int64
ITEM DESCRIPTION	0	SUPPLIER	object
ITEM TYPE	0	ITEM CODE	object
RETAIL SALES	1	ITEM DESCRIPTION	object
RETAIL TRANSFERS	0	ITEM TYPE	object
WAREHOUSE SALES	0	RETAIL SALES	float64
dtype: int64		RETAIL TRANSFERS	float64
		WAREHOUSE SALES	float64

```
'CROOK & MARKER LLC' 'SNR HOLDINGS LLC' 'PARALLEL WORLD BREWING COMPANY'
'RSL HOLDINGS INC' 'GJS SALES INC'
'VINIFERA DISTRIBUTING OF MARYLAND INC' 'MARQUEE SELECTIONS LLC'
'SMITH-ANDERSON ENTERPRISES INC']
```

8. EXPLORATORY DATA ANALYSIS

Exploratory Data Analysis (EDA) is performed to understand the structure, characteristics and patterns present in the Retail and warehouse Sale.csv dataset. The dataset is explored using Python and suitable visualizations.

CODE:

A. Basic EDA

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
# Load dataset
df = pd.read_csv("Retail and warehouse Sale.csv")
# First five records
print("First Five Records:")
print(df.head())
# Last five records
print("\nLast Five Records:")
print(df.tail())
# Dataset shape
print("\nShape:")
print(df.shape)
# Column names
print("\nColumn Names:")
print(df.columns)
# Data types
print("\nData Types:")
print(df.dtypes)
# Missing values
print("\nMissing Values:")
print(df.isnull().sum())
# Duplicate records
print("\nDuplicate Records:")
print(df.duplicated().sum())
# Unique values
print("\nUnique Values:")
print(df.nunique())
# Basic statistics
print("\nBasic Statistics:")
print(df.describe())
```

OUTPUT:

First Five Records:

	YEAR	MONTH	SUPPLIER	ITEM CODE	\
0	2020	1	REPUBLIC NATIONAL DISTRIBUTING CO	100009	
1	2020	1	PWSWN INC	100024	
2	2020	1	RELIABLE CHURCHILL LLLP	1001	
3	2020	1	LANTERNA DISTRIBUTORS INC	100145	
4	2020	1	DIONYSOS IMPORTS INC	100293	

	ITEM DESCRIPTION	ITEM TYPE	RETAIL SALES	\
0	BOOTLEG RED - 750ML	WINE	0.00	
1	MOMENT DE PLAISIR - 750ML	WINE	0.00	
2	S SMITH ORGANIC PEAR CIDER - 18.7OZ	BEER	0.00	
3	SCHLINK HAUS KABINETT - 750ML	WINE	0.00	
4	SANTORINI GAVALA WHITE - 750ML	WINE	0.82	

	RETAIL TRANSFERS	WAREHOUSE SALES
0	0.0	2.0
1	1.0	4.0
2	0.0	1.0
3	0.0	1.0
4	0.0	0.0

Last Five Records:

	YEAR	MONTH	SUPPLIER ITEM CODE \		
29995	2020	3	THE COUNTRY VINTNER, LLC DBA WINEBOW 352322		
...					
25%	2020.0	1.000000	0.000000	0.000000	0.000000
50%	2020.0	3.000000	0.160000	0.000000	1.000000
75%	2020.0	7.000000	2.920000	3.000000	6.000000
max	2020.0	9.000000	2739.000000	1507.000000	18317.000000

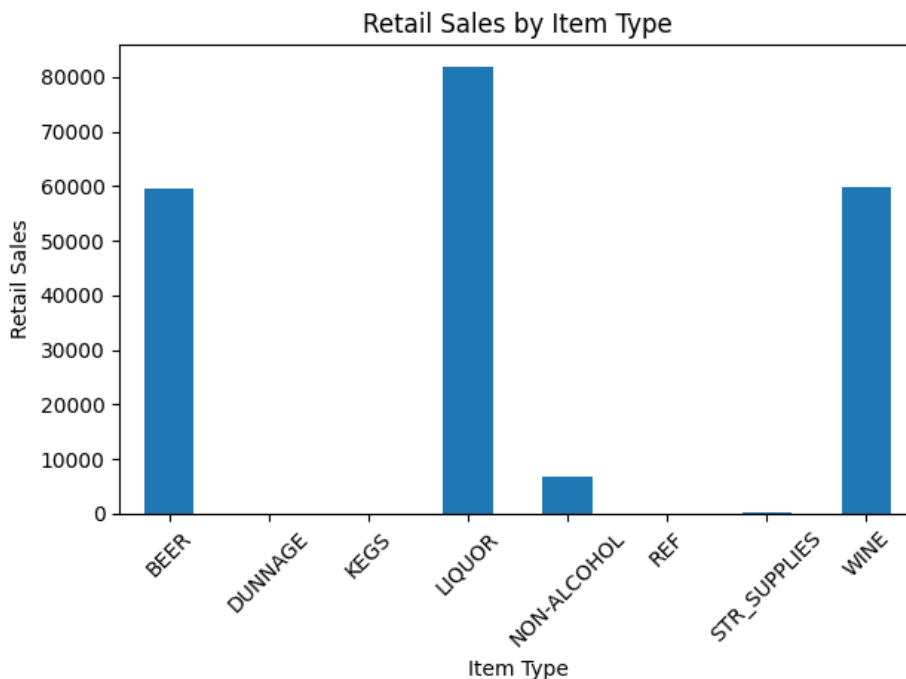
B. EDA Charts

1. Bar Chart – Sales by Item Type

CODE:

```
df.groupby("ITEM TYPE")["RETAIL SALES"].sum().plot(kind="bar")
plt.title("Retail Sales by Item Type")
plt.xlabel("Item Type")
plt.ylabel("Retail Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

OUTPUT:



Observation:

This chart compares total retail sales across different item types and helps identify the item types contributing more to retail sales.

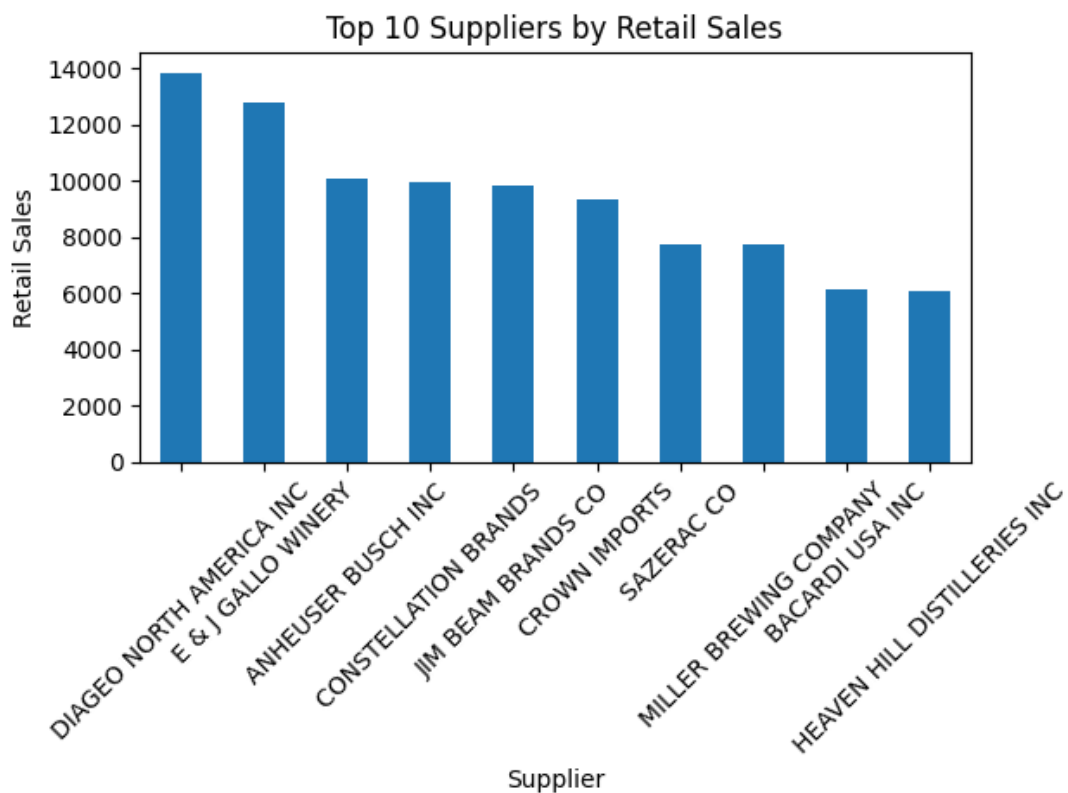
2. Bar Chart – Sales by Supplier

CODE:

```
df.groupby("SUPPLIER")["RETAIL SALES"].sum().nlargest(10).plot(kind="bar")
plt.title("Top 10 Suppliers by Retail Sales")
plt.xlabel("Supplier")
plt.ylabel("Retail Sales")
plt.xticks(rotation=45)
```

```
plt.tight_layout()
plt.show()
```

OUTPUT:



Observation:

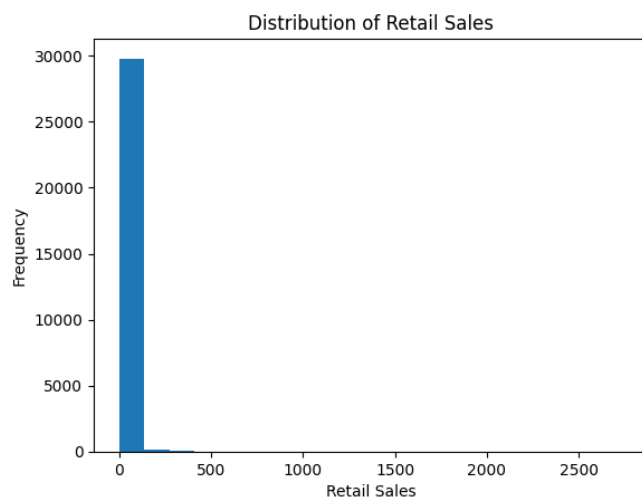
The chart compares the retail sales associated with the top suppliers and helps understand supplier-wise sales performance.

3. Histogram – Retail Sales

CODE:

```
plt.hist(df["RETAIL SALES"], bins=20)
plt.title("Distribution of Retail Sales")
plt.xlabel("Retail Sales")
plt.ylabel("Frequency")
plt.show()
```

OUTPUT:

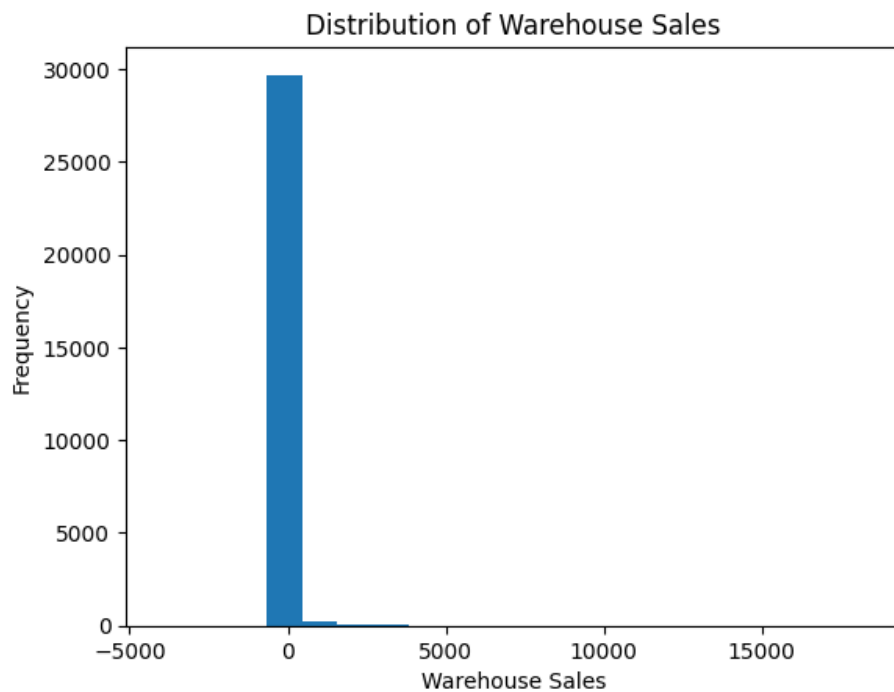


Observation:

The histogram shows the distribution of retail sales values and helps understand the spread and concentration of sales.

4. Histogram – Warehouse Sales**CODE:**

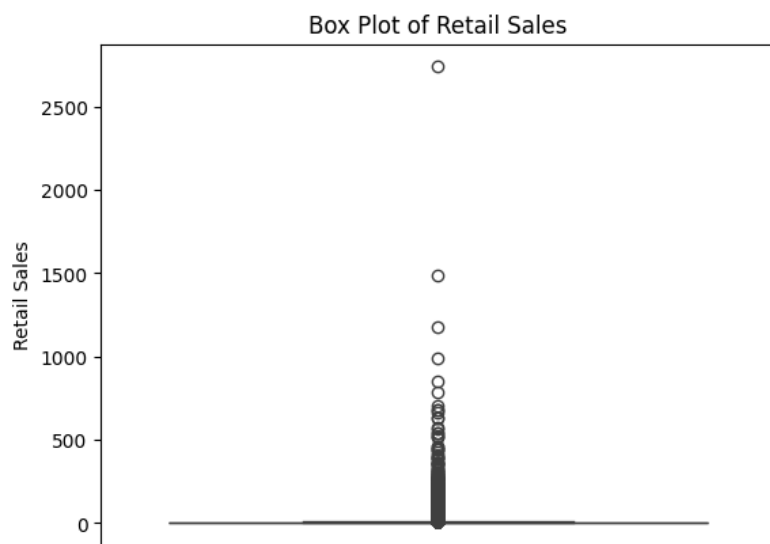
```
plt.hist(df["WAREHOUSE SALES"], bins=20)
plt.title("Distribution of Warehouse Sales")
plt.xlabel("Warehouse Sales")
plt.ylabel("Frequency")
plt.show()
```

OUTPUT:**Observation:**

The histogram shows how warehouse sales values are distributed across the dataset.

5. Box Plot – Retail Sales**CODE:**

```
sns.boxplot(y=df["RETAIL SALES"])
plt.title("Box Plot of Retail Sales")
plt.ylabel("Retail Sales")
plt.show()
```

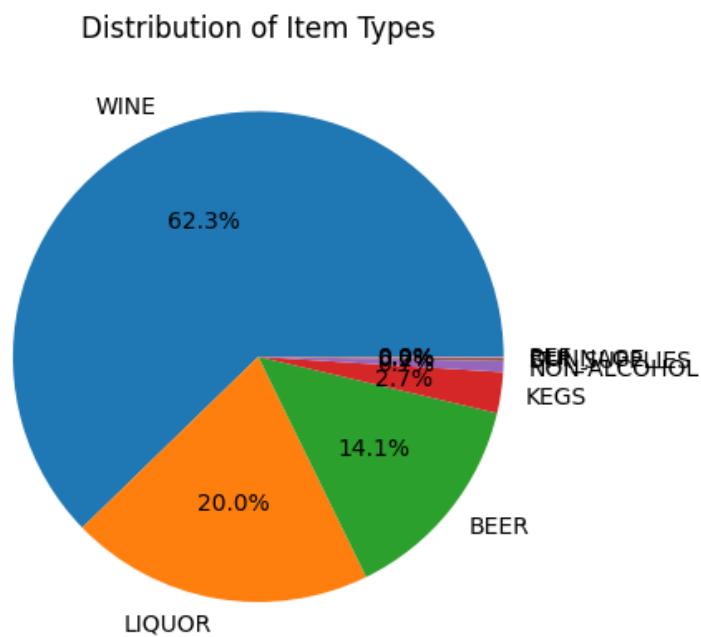
OUTPUT:

Observation:

The box plot shows the median, spread and possible outliers in retail sales.

6. Pie Chart – Item Type**CODE:**

```
df["ITEM TYPE"].value_counts().head(10).plot(
    kind="pie",
    autopct="%1.1f%%"
)
plt.title("Distribution of Item Types")
plt.ylabel("")
plt.show()
```

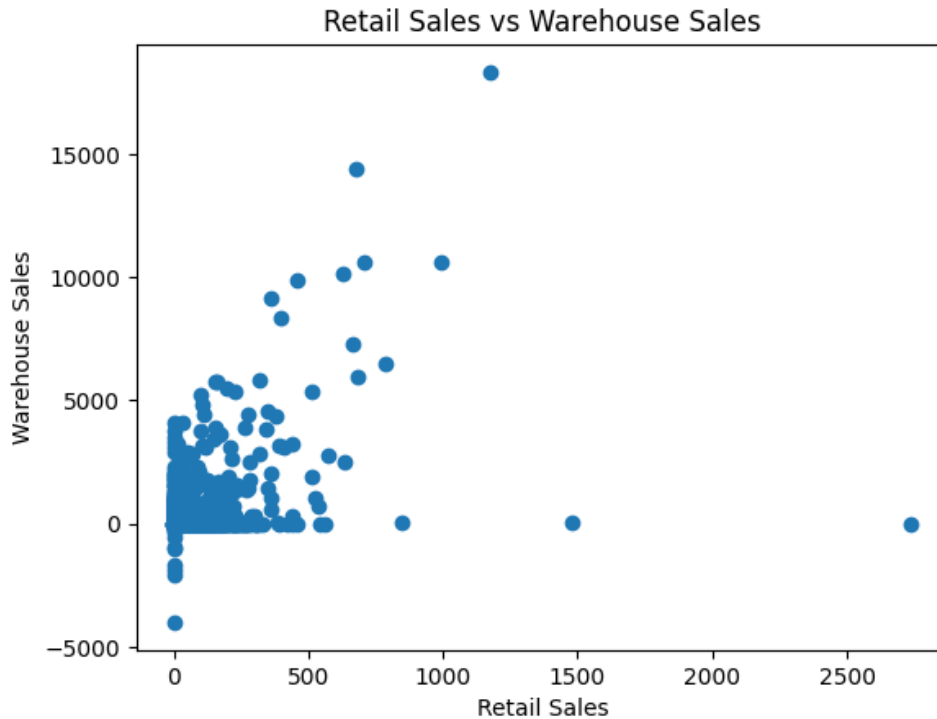
OUTPUT:**Observation:**

The pie chart shows the proportion of records belonging to the different item types.

7. Scatter Plot – Retail Sales vs Warehouse Sales**CODE:**

```
plt.scatter(df["RETAIL SALES"], df["WAREHOUSE SALES"])
plt.title("Retail Sales vs Warehouse Sales")
plt.xlabel("Retail Sales")
plt.ylabel("Warehouse Sales")
plt.show()
```

OUTPUT:



Observation:

The scatter plot helps identify whether there is a relationship between retail sales and warehouse sales.

C. EDA Summary

The EDA helps to understand the structure, distribution and important patterns in the retail and warehouse sales data. The charts provide a visual comparison of item types and suppliers, show the distribution of sales values, identify possible outliers and help examine the relationship between retail and warehouse sales.

Important: For your final report, write the observations based on your actual chart outputs. For example, don't say that a particular item type has the highest sales until your generated chart confirms it.

9. DESCRIPTIVE STATISTICAL ANALYSIS

Descriptive statistical analysis is performed to summarize the important numerical variables in the Retail and warehouse Sale.csv dataset. The selected variables are RETAIL SALES, RETAIL TRANSFERS and WAREHOUSE SALES.

CODE:

```
import pandas as pd
# Load dataset
df = pd.read_csv("Retail and warehouse Sale.csv")
# Select numerical variables
columns = ["RETAIL SALES", "RETAIL TRANSFERS", "WAREHOUSE SALES"]
# Calculate descriptive statistics
for col in columns:
    print("\n", col)
    print("Mean:", df[col].mean())
    print("Median:", df[col].median())
    print("Mode:", df[col].mode()[0])
    print("Minimum:", df[col].min())
```

```
print("Maximum:", df[col].max())
print("Range:", df[col].max() - df[col].min())
print("Variance:", df[col].var())
print("Standard Deviation:", df[col].std())
```

OUTPUT:

RETAIL SALES

```
Mean: 6.939795993199773
Median: 0.16
Mode: 0.0
Minimum: -0.42
Maximum: 2739.0
Range: 2739.42
Variance: 1094.3561198730408
Standard Deviation: 33.08105379024436
```

RETAIL TRANSFERS

```
Mean: 6.5940579999999995
Median: 0.0
Mode: 0.0
Minimum: -6.0
Maximum: 1507.0
Range: 1513.0
Variance: 777.2625258834987
Standard Deviation: 27.879428363642944
```

WAREHOUSE SALES

```
Mean: 27.431030999999997
Median: 1.0
Mode: 0.0
...
Maximum: 18317.0
Range: 22316.0
Variance: 74074.37769347016
Standard Deviation: 272.16608475978444
```

Interpretation

- Mean shows the average value of each sales variable.
- Median represents the middle value and helps understand the typical observation.
- Mode shows the most frequently occurring value.
- Minimum and Maximum show the lowest and highest recorded values.
- Range shows the overall spread between the minimum and maximum values.
- Variance measures how much the values differ from their mean.
- Standard deviation indicates the amount of variation or spread in the data.

Overall, these statistics help understand the central tendency and variability of retail sales, retail transfers and warehouse sales, which provides a basis for further business analysis.

10. DISTRIBUTION ANALYSIS

For distribution analysis, we will use **RETAIL SALES** as the important numerical variable.

Aim

To study the distribution of retail sales using a **histogram** and **box plot**, and understand its shape, central tendency, spread, skewness and outliers.

CODE:

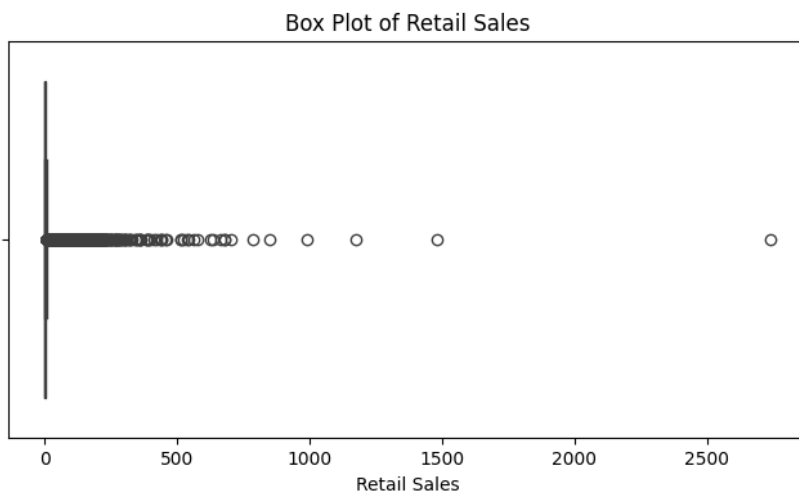
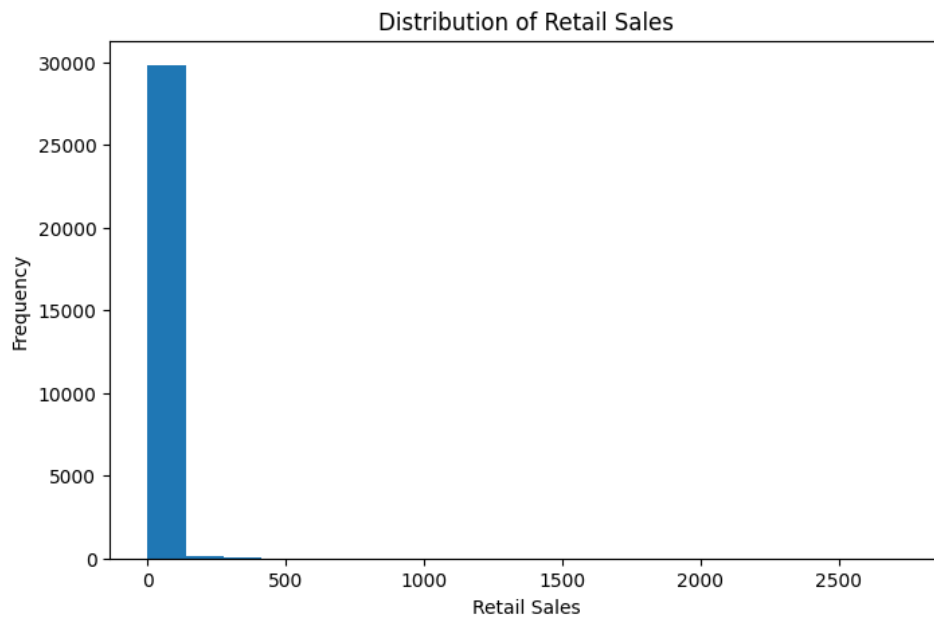
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
df = pd.read_csv("Retail and warehouse Sale.csv")
```

```

# Histogram
plt.figure(figsize=(8,5))
plt.hist(df["RETAIL SALES"].dropna(), bins=20)
plt.title("Distribution of Retail Sales")
plt.xlabel("Retail Sales")
plt.ylabel("Frequency")
plt.show()
# Box Plot
plt.figure(figsize=(8,4))
sns.boxplot(x=df["RETAIL SALES"])
plt.title("Box Plot of Retail Sales")
plt.xlabel("Retail Sales")
plt.show()
# Skewness
print("Skewness:", df["RETAIL SALES"].skew())

```

OUTPUT:



Skewness: 30.23069622297996

Analysis:

- Shape of distribution: The histogram shows how the retail sales values are distributed across different ranges.
- Central tendency: The center of the distribution can be understood using the mean and median of retail sales.
- Spread: The width of the distribution shows the variation in retail sales values. The box plot also shows the interquartile range.
- Outliers: Values appearing separately beyond the whiskers of the box plot can be considered potential outliers.
- Skewness: The skewness value indicates whether the distribution is approximately symmetric or tilted toward one side. A positive value indicates right skewness, while a negative value indicates left skewness.

Short Interpretation:

The histogram and box plot were used to study the distribution of RETAIL SALES. The histogram shows the frequency and shape of the sales data, while the box plot shows the central spread and potential outliers. The skewness value helps identify whether the data is symmetric or skewed. This analysis helps understand the variation in retail sales and identify unusual sales values that may require further investigation.

11. CORRELATION ANALYSIS

Aim

To study the relationships between important numerical variables in the **Retail and Warehouse Sales** dataset using a correlation matrix and heatmap.

Numerical Variables Selected

1. RETAIL SALES
2. RETAIL TRANSFERS
3. WAREHOUSE SALES

CODE:

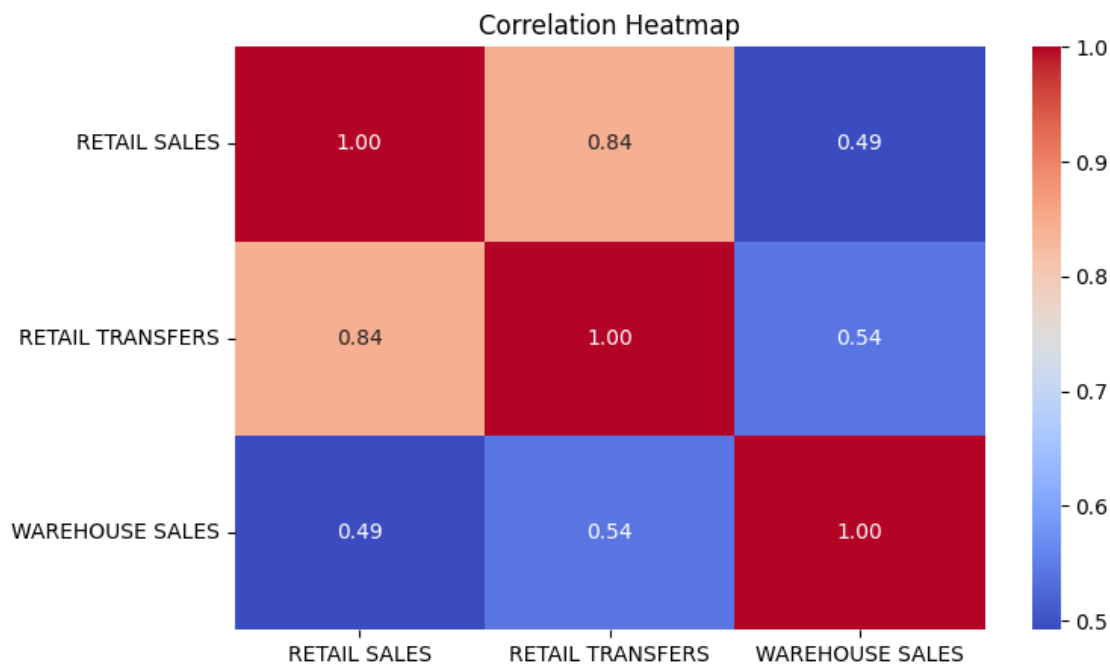
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
df = pd.read_csv("Retail and wherehouse Sale.csv")

# Select numerical variables
data = df[["RETAIL SALES", "RETAIL TRANSFERS", "WAREHOUSE SALES"]]
# Correlation matrix
correlation = data.corr()
print("Correlation Matrix:")
print(correlation)
# Heatmap
plt.figure(figsize=(8,5))
sns.heatmap(correlation, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()
```

OUTPUT:

Correlation Matrix:

	RETAIL SALES	RETAIL TRANSFERS	WAREHOUSE SALES
RETAIL SALES	1.000000	0.844597	0.491822
RETAIL TRANSFERS	0.844597	1.000000	0.540071
WAREHOUSE SALES	0.491822	0.540071	1.000000



Analysis:

After running the code, examine the correlation matrix and identify:

- Positive relationship: When one variable increases, the other tends to increase.
- Negative relationship: When one variable increases, the other tends to decrease.
- Strong relationship: Correlation value is close to +1 or -1.
- Weak relationship: Correlation value is close to 0.

Business Interpretation:

- Retail Sales and Warehouse Sales: A positive correlation would indicate that higher warehouse sales are generally associated with higher retail sales. This can help the business understand the connection between warehouse activity and retail performance.
- Retail Sales and Retail Transfers: The correlation indicates whether product transfers are associated with retail sales. A positive relationship may suggest that higher transfers support higher retail sales.
- Retail Transfers and Warehouse Sales: This relationship can help understand whether warehouse activity and product transfers move together.

Short Conclusion:

Correlation analysis helps identify relationships between retail sales, retail transfers and warehouse sales. The correlation matrix and heatmap make it easy to identify positive, negative, strong and weak relationships. These relationships can help the business understand sales and warehouse activities and support better inventory and sales planning.

12. REGRESSION ANALYSIS

- **Independent Variable (X):** RETAIL TRANSFERS
- **Dependent Variable (Y):** RETAIL SALES

This checks whether retail transfers are linearly related to retail sales.

CODE:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
df = pd.read_csv("Retail and warehouse Sale.csv")

data = df[["RETAIL TRANSFERS", "RETAIL SALES"]].dropna()
x = data["RETAIL TRANSFERS"].values
y = data["RETAIL SALES"].values
```

```

# Regression coefficient and intercept
coefficient = np.cov(x, y, bias=True)[0, 1] / np.var(x)
intercept = y.mean() - coefficient * x.mean()
# Predicted values
y_pred = intercept + coefficient * x
# R2
r2 = 1 - np.sum((y - y_pred) ** 2) / np.sum((y -
y.mean()) ** 2)
# MSE and RMSE
mse = np.mean((y - y_pred) ** 2)
rmse = np.sqrt(mse)
print("Regression Equation:")
print("Retail Sales =", intercept, "+", coefficient, "*
Retail Transfers")
print("\nRegression Coefficient:", coefficient)
print("R2 Score:", r2)

```

OUTPUT:

```

Regression Equation:
Retail Sales = 0.3312546053743137 + 1.0021630236058785 * Retail Transfers

```

```

Regression Coefficient: 1.0021630236058785
R2 Score: 0.7133447600155359
MSE: 313.69245905823067
RMSE: 17.71136525111011

```

```

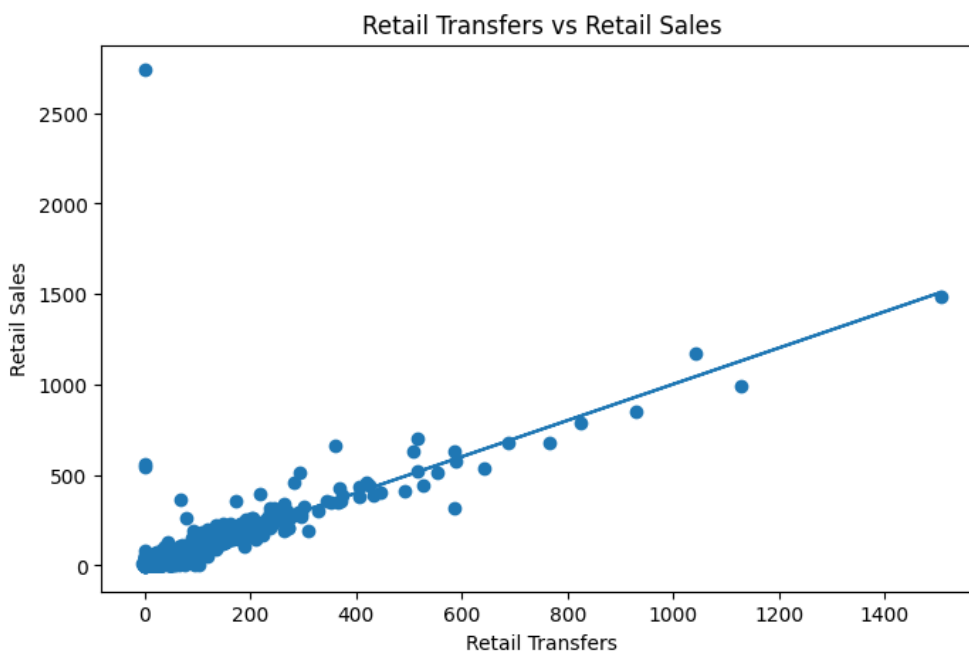
Predicted Retail Sales for 100 Retail Transfers: 100.54755696596216

```

```

print("MSE:", mse)
print("RMSE:", rmse)
# Prediction
transfer_value = 100
prediction = intercept + coefficient * transfer_value
print("\nPredicted Retail Sales for 100 Retail
Transfers:", prediction)
# Scatter plot and regression line
plt.figure(figsize=(8,5))
plt.scatter(x, y)
plt.plot(x, y_pred)
plt.title("Retail Transfers vs Retail Sales")
plt.xlabel("Retail Transfers")
plt.ylabel("Retail Sales")
plt.show()

```



Interpretation:

- Regression coefficient: Shows the expected change in retail sales for a one-unit increase in retail transfers.
- R² score: Shows how much of the variation in retail sales is explained by retail transfers. A value closer to 1 indicates greater explanatory power.
- MSE: Measures the average squared prediction error.

- RMSE: Measures prediction error in the same units as retail sales. Lower RMSE indicates smaller prediction errors.
- Regression line: Shows the overall linear relationship between retail transfers and retail sales.

Business Interpretation:

If the regression coefficient is positive, higher retail transfers are associated with higher retail sales. This can help the business understand whether product movement through retail transfers is associated with sales performance.

The R^2 score indicates how useful retail transfers are for explaining variation in retail sales. The prediction can be used to estimate retail sales for a given level of retail transfers.

Short Conclusion:

Linear regression was performed to study the relationship between Retail Transfers and Retail Sales. The regression coefficient, R^2 score, MSE and RMSE were calculated to evaluate the relationship and prediction accuracy. The scatter plot with the regression line provides a visual representation of the relationship between the two variables.

13. BUSINESS INTELLIGENCE DASHBOARD

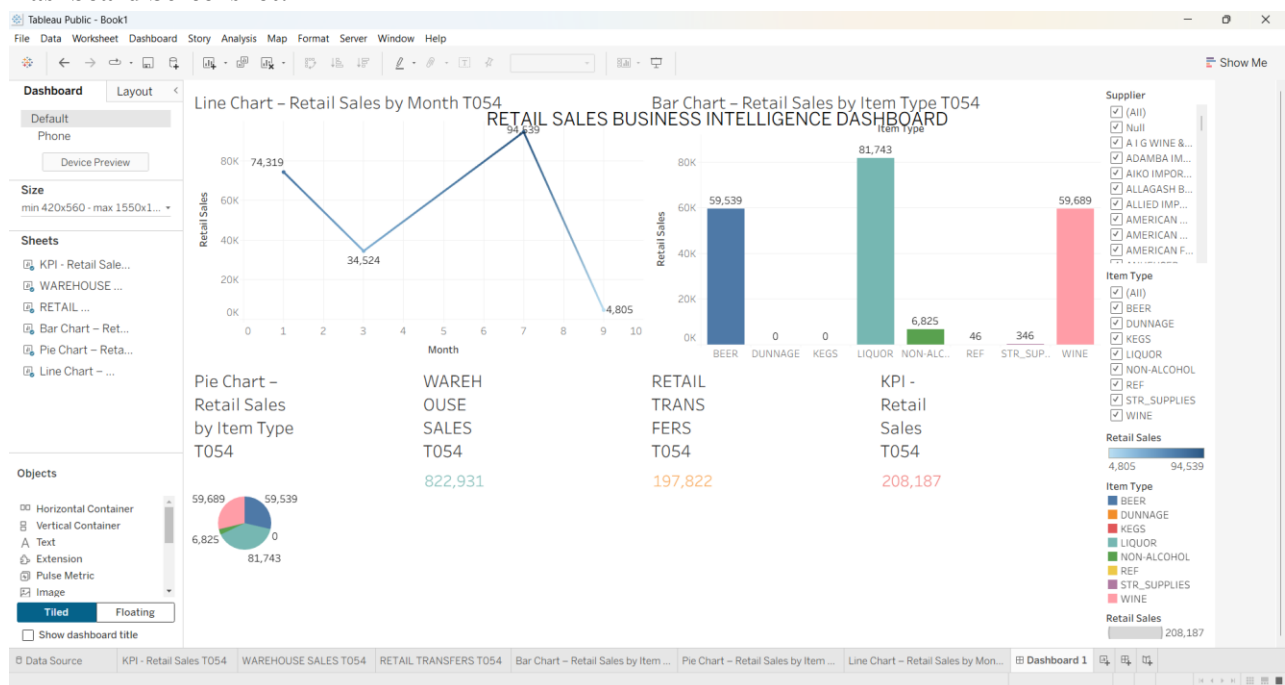
Aim:

To create an interactive Business Intelligence dashboard to analyze retail sales, warehouse sales, suppliers and item types and obtain useful business insights.

Dashboard Components:

Component	Representations
KPI Card 1	Total Retail Sales
KPI Card 2	Total Warehouse Sales
KPI Card 3	Total Retail Transfers
Bar Chart	Retail Sales by Item Type
Pie Chart	Retail Sales by Item Type
Line Chart	Retail Sales by Month
Filter 1	Item Type
Filter 2	Supplier
Filter 3	Month
Titles	Clear titles for every chart

Dashboard Screenshot:



Five Business Insights:

1. Retail Sales by Item Type: Different item types contribute different amounts to total retail sales, helping identify categories with higher and lower sales.
2. Monthly Sales: The monthly line chart shows changes in retail sales across different months and helps identify periods of relatively higher or lower sales.
3. Warehouse Sales: Total warehouse sales provide information about the level of warehouse-related sales activity.
4. Retail Transfers: Retail transfers show the movement of products through retail operations and can be compared with retail sales to understand their relationship.
5. Supplier Performance: The supplier filter allows the business to examine sales associated with individual suppliers and compare their contribution to retail sales.

Final Conclusion:

The Business Intelligence dashboard provides an interactive view of retail and warehouse sales data. KPI cards summarize important business measures, while bar, pie and line charts provide comparisons and trends. Filters allow users to analyze the data by item type, supplier and month. The dashboard helps the business understand sales performance and supports better sales, inventory and warehouse-related decision-making.

14. PROBABILITY AND PROBABILITY DISTRIBUTIONS

Aim:

To perform a simple probability experiment using Python and generate random values from Normal, Uniform and Exponential distributions. The characteristics of each distribution are studied using graphs.

A. Probability Experiment – Dice Rolling:

simulate 1000 rolls of a fair six-sided die and calculate the probability of getting each outcome.

CODE:

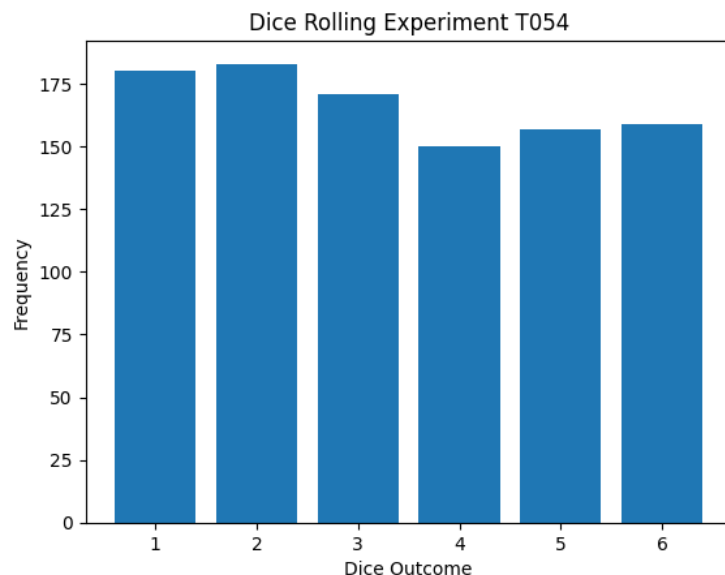
```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(10)
# Simulate 1000 dice rolls
rolls = np.random.randint(1, 7, 1000)
# Probability of each outcome
for i in range(1, 7):
    probability = np.sum(rolls == i) / len(rolls)
    print("Probability of", i, "=", probability)
# Probability of getting more than 4
```

```
p = np.sum(rolls > 4) / len(rolls)
print("\nProbability of getting more than 4:", p)
# Graph
plt.hist(rolls, bins=np.arange(1, 8)-0.5, rwidth=0.8)
plt.title("Dice Rolling Experiment T054")
plt.xlabel("Dice Outcome")
plt.ylabel("Frequency")
plt.xticks(range(1, 7))
plt.show()
```

OUTPUT:

```
Probability of 1 = 0.18
Probability of 2 = 0.183
Probability of 3 = 0.171
Probability of 4 = 0.15
Probability of 5 = 0.157
Probability of 6 = 0.159

Probability of getting more than 4: 0.316
```



Interpretation:

The probability of each dice outcome should be approximately $\frac{1}{6}$ (**0.167**) when a large number of rolls are performed. Small differences occur because the experiment is random.

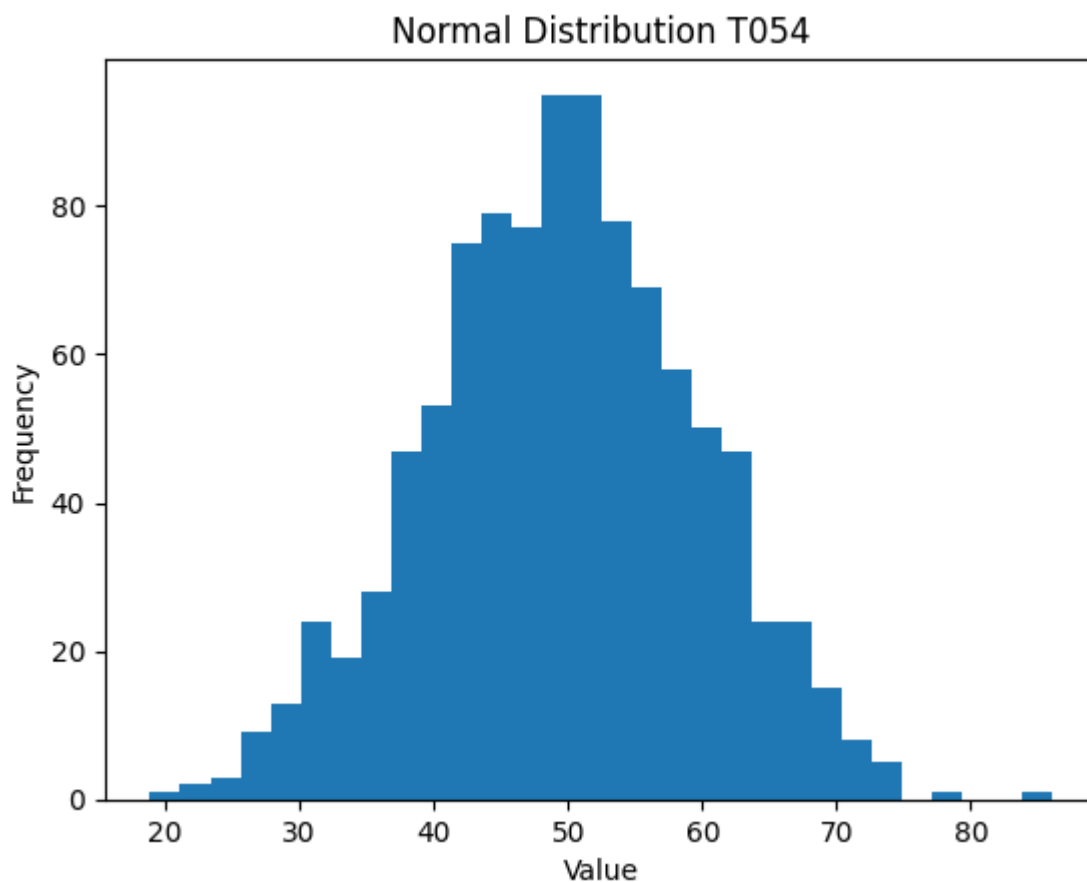
B. Normal Distribution

A normal distribution is a bell-shaped and approximately symmetric distribution where most values occur around the mean.

CODE:

```
# Normal distribution
normal_data = np.random.normal(50, 10, 1000)
plt.hist(normal_data, bins=30)
plt.title("Normal Distribution T054")

plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```

OUTPUT:**Characteristics**

- Bell-shaped distribution
- Approximately symmetric
- Values are concentrated around the mean
- Mean, median and mode are approximately equal

C. Uniform Distribution:

In a uniform distribution, values within a specified range have approximately equal chances of occurring.

CODE:

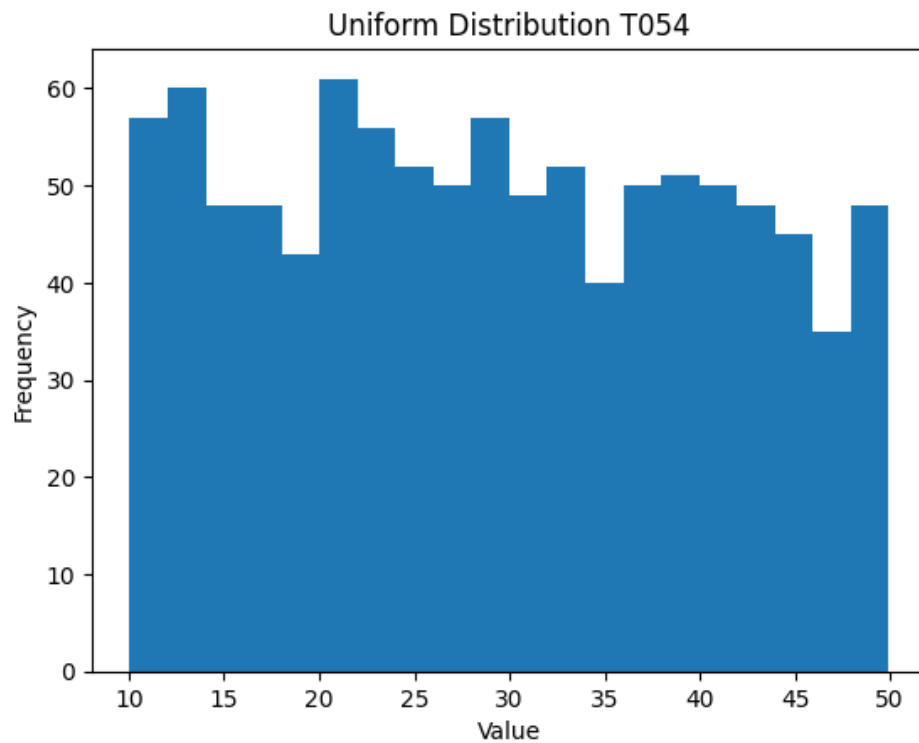
```
# Uniform distribution
uniform_data = np.random.uniform(10, 50, 1000)

plt.hist(uniform_data, bins=20)
plt.title("Uniform Distribution T054")
```

```
plt.xlabel("Value")
plt.ylabel("Frequency")
```

```
plt.show()
```

OUTPUT:



Characteristics

- Values are generated within a fixed range.
- Values have approximately equal probability.
- The graph has a relatively flat shape.
- Useful for representing random values where no value is preferred.

D. Exponential Distribution

The exponential distribution is commonly used to represent waiting time or time between events.

CODE:

```
# Exponential distribution
```

```
exponential_data = np.random.exponential(10, 1000)
```

```
plt.hist(exponential_data, bins=30)
```

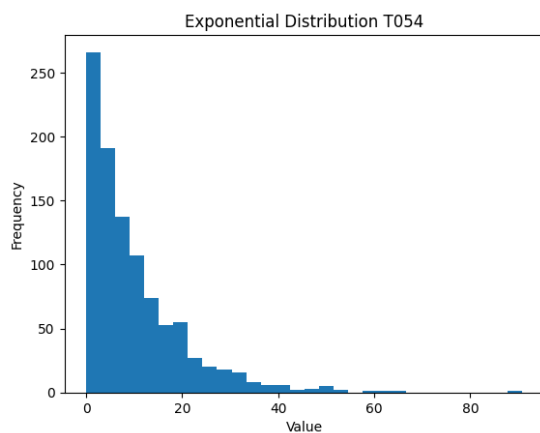
```
plt.title("Exponential Distribution T054")
```

```
plt.xlabel("Value")
```

```
plt.ylabel("Frequency")
```

```
plt.show()
```

OUTPUT:



Characteristics

- Right-skewed distribution
- Many values occur near zero.
- Frequency decreases as the value increases.
- Useful for studying waiting times and time between events.

Business Interpretation

Probability and probability distributions can help a retail business understand uncertainty and random variation. For example, probability can be used to study the likelihood of different demand levels, while distributions can be used to model sales, waiting times and other business-related uncertainties.

Final Conclusion

The dice experiment demonstrated how probabilities can be estimated using repeated random trials. Normal, Uniform and Exponential distributions were generated using Python and visualized using histograms. The Normal distribution is approximately symmetric, the Uniform distribution is relatively flat, and the Exponential distribution is right-skewed. These concepts are useful for understanding uncertainty and variability in business data.

15. DECISION MAKING UNDER UNCERTAINTY

Aim:

To analyze a retail business decision under uncertainty using alternatives, possible outcomes, probabilities, payoffs and expected value.

Business Scenario:

ABC Retail Store wants to decide how much inventory to purchase for an upcoming month. Demand is uncertain, so the business has two alternatives:

- **Alternative A:** Purchase a large quantity of inventory.
- **Alternative B:** Purchase a small quantity of inventory.

The business expects either High Demand or Low Demand.

1. Alternatives

Alternative	Decision
A	Purchase Large Inventory
B	Purchase Small Inventory

2. Possible Outcomes

- High Demand
- Low Demand

3. Probabilities and Payoffs

Alternative	High Demand (0.6)	Low Demand (0.4)
Large Inventory	₹80,000	₹20,000
Small Inventory	₹50,000	₹35,000

The probabilities are assumed based on the expected demand conditions for this business decision.

CODE:

```
# Decision Making Under Uncertainty
alternatives = {
    "Large Inventory": [(0.6, 80000), (0.4, 20000)],
    "Small Inventory": [(0.6, 50000), (0.4, 35000)]
}

for alternative, outcomes in alternatives.items():
    expected_value = 0
    for probability, payoff in outcomes:
        expected_value += probability * payoff
    print(alternative)
```

```

print("Expected Value:", expected_value)
print()
# Compare alternatives
ev_large = 0.6 * 80000 + 0.4 * 20000
ev_small = 0.6 * 50000 + 0.4 * 35000
print("Comparison:")

```

OUTPUT:

```

Large Inventory
Expected Value: 56000.0

```

```

Small Inventory
Expected Value: 44000.0

```

```

Comparison:
Large Inventory EV: 56000.0
Small Inventory EV: 44000.0

```

```

Higher Expected Value: Large Inventory

```

```

print("Large Inventory EV:", ev_large)
print("Small Inventory EV:", ev_small)
if ev_large > ev_small:
    print("\nHigher Expected Value: Large Inventory")
else:
    print("\nHigher Expected Value: Small Inventory")

```

16. SAMPLING AND SAMPLING DISTRIBUTION

Aim

To select random samples from the retail sales dataset, calculate sample means repeatedly, study the sampling distribution, and understand the Central Limit Theorem (CLT).

1. Population:

The population is all the observations of RETAIL SALES available in Retail and warehouse Sale.csv.

2. Sample:

A sample is a randomly selected subset of retail sales observations from the population.

3. Sample Sizes:

We will compare three sample sizes:

- 30
- 50
- 100

For each sample size, 100 random samples will be selected.

4. Sampling Technique:

Simple Random Sampling is used, where every observation has an equal chance of being selected.

CODE:

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
df = pd.read_csv("Retail and warehouse Sale.csv")
# Population
population = df["RETAIL SALES"].dropna().values
print("Population Size:", len(population))
np.random.seed(10)
sample_sizes = [30, 50, 100]
for size in sample_sizes:
    sample_means = []
    # Take 100 random samples
    for i in range(100):
        sample = np.random.choice(population,
size=size, replace=False)

```

```

        sample_means.append(sample.mean())
    sample_means = np.array(sample_means)
    mean_of_means = sample_means.mean()
    sd = sample_means.std(ddof=1)
    standard_error = population.std(ddof=1) /
np.sqrt(size)
    print("\nSample Size:", size)
    print("Mean of Sample Means:", mean_of_means)
    print("Standard Deviation:", sd)
    print("Standard Error:", standard_error)
    # Histogram
    plt.figure(figsize=(8,5))
    plt.hist(sample_means, bins=15)
    plt.title("Sampling Distribution of Sample Means
T054 - Sample Size " + str(size))

```

```
plt.xlabel("Sample Mean")  
plt.ylabel("Frequency")  
plt.show()
```

OUTPUT:

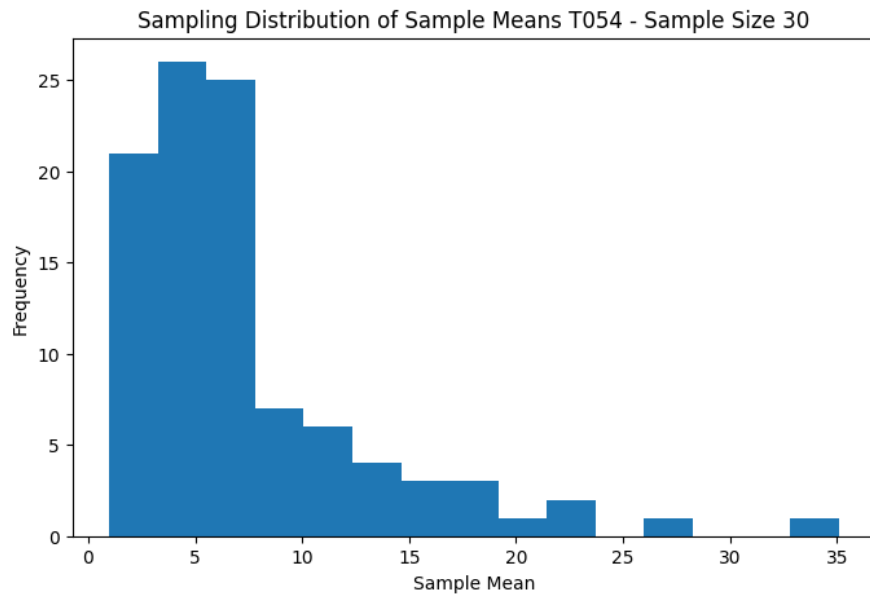
Population Size: 29999

Sample Size: 30

Mean of Sample Means: 7.386556666666669

Standard Deviation: 5.778625934649502

Standard Error: 6.0397464623195365

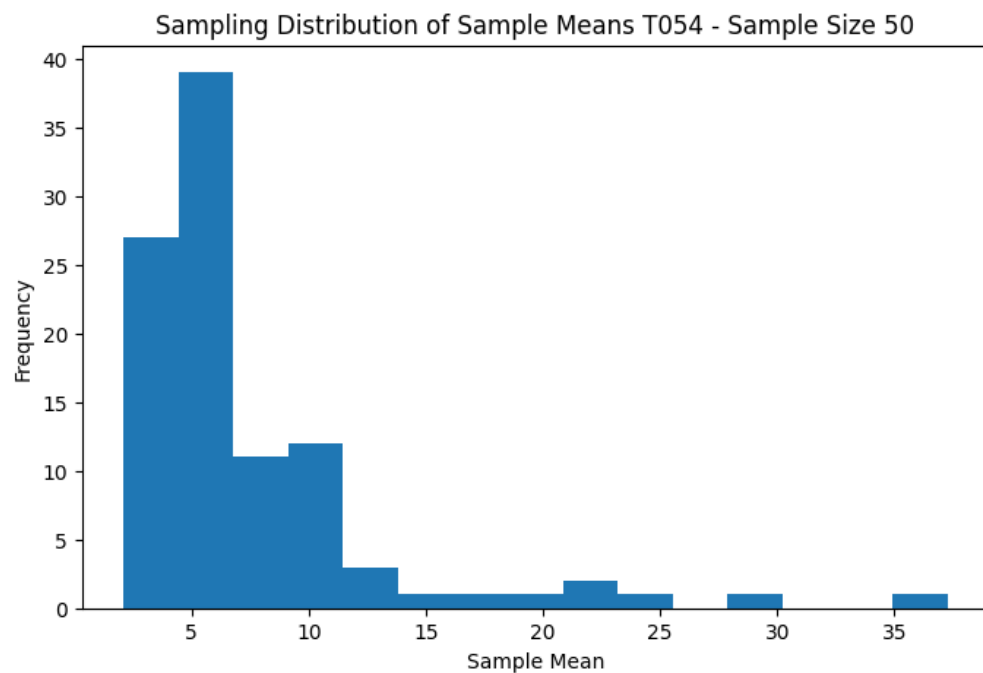


Sample Size: 50

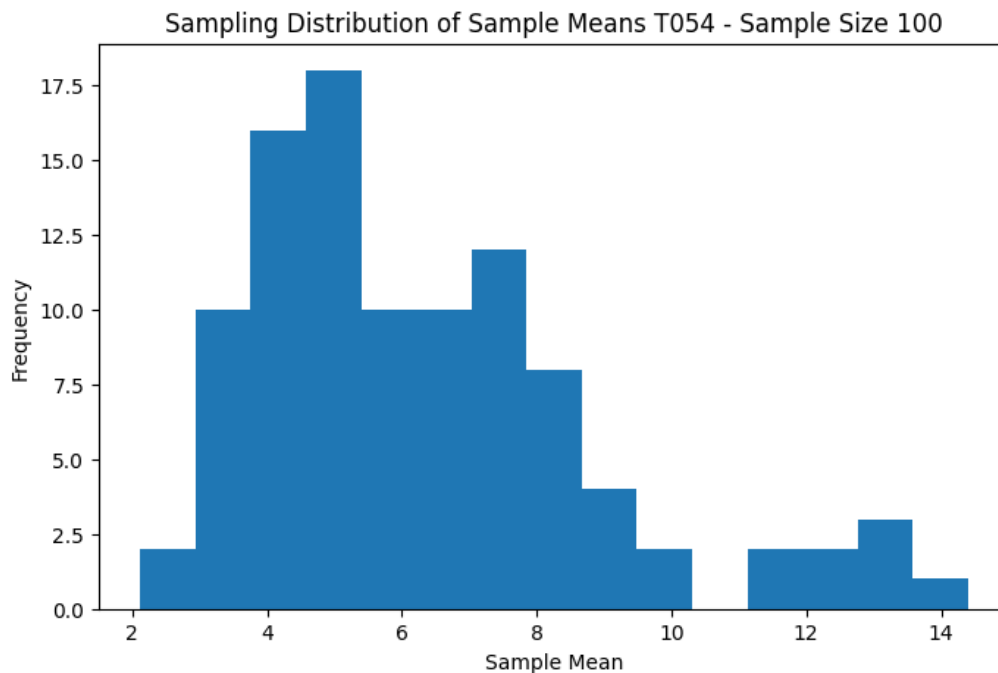
Mean of Sample Means: 7.402827999999999

Standard Deviation: 5.681753830741655

Standard Error: 4.678367492775745



Sample Size: 100
Mean of Sample Means: 6.344690999999999
Standard Deviation: 2.521576613389113
Standard Error: 3.308105379024436



Observed:

- The mean of sample means should generally be close to the population mean.
- As the sample size increases, the sample means generally become less variable.
- Therefore, the standard error decreases as sample size increases.
- The histogram of sample means becomes more concentrated around the population mean.

Central Limit Theorem:

The Central Limit Theorem (CLT) states that when sufficiently large random samples are repeatedly taken from a population, the distribution of their sample means tends to become approximately normal, even if the original population is not normally distributed.

In this practical, repeated samples of sizes 30, 50 and 100 are taken. Their sample means form sampling distributions. As the sample size increases, the distribution of sample means becomes more concentrated around the population mean.

Business Interpretation:

Sampling allows a business to estimate the overall retail sales performance without analyzing every observation each time. A larger sample generally provides a more stable estimate of the population mean and reduces sampling variation.

Conclusion:

Simple random sampling was performed on RETAIL SALES using 100 repeated samples for three different sample sizes. The mean of sample means, standard deviation and standard error were calculated. The results demonstrate that increasing the sample size reduces the variation of sample means, supporting the Central Limit Theorem.

17. CONFIDENCE INTERVAL ESTIMATION

Calculate confidence intervals for the mean RETAIL SALES of the selected retail dataset.

Objective:

To estimate the population mean of Retail Sales using:

- 90% Confidence Interval
- 95% Confidence Interval
- 99% Confidence Interval

CODE:

```
import pandas as pd
import numpy as np
from scipy import stats
df = pd.read_csv("Retail and warehouse Sale.csv")
sales = df["RETAIL SALES"].dropna()
n = len(sales)
mean = sales.mean()
std = sales.std()
confidence_levels = [0.90, 0.95, 0.99]
print("Mean Retail Sales:", mean)
```

OUTPUT:

Mean Retail Sales: 6.939795993199773

Confidence Level: 90 %
Lower Limit: 6.625624616460265
Upper Limit: 7.253967369939281

Confidence Level: 95 %
Lower Limit: 6.565434163855106
Upper Limit: 7.31415782254444

Confidence Level: 99 %
Lower Limit: 6.447789697214849
Upper Limit: 7.431802289184698

```
for confidence in confidence_levels:
```

```
    alpha = 1 - confidence
    t_value = stats.t.ppf(1 - alpha/2, n - 1)
    margin_error = t_value * (std / np.sqrt(n))
    lower = mean - margin_error
    upper = mean + margin_error
    print("\nConfidence Level:", int(confidence * 100),
          "%")
    print("Lower Limit:", lower)
    print("Upper Limit:", upper)
```

Interpretation:

The 95% confidence interval gives a range of values within which the true population mean of Retail Sales is estimated to lie, based on the sample data.

In the business context, this interval helps the retail business estimate the average retail sales level without having to rely only on the sample mean. It can support decisions related to sales planning, inventory management and future business planning.

18. BOOTSTRAPPING

we will use RETAIL SALES as the numerical variable and calculate the median retail sales using bootstrapping.

Objective:

To estimate the 95% bootstrap confidence interval for the median of Retail Sales by repeatedly taking random samples with replacement.

CODE:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
df = pd.read_csv("Retail and warehouse Sale.csv")
sales = df["RETAIL SALES"].dropna().values
np.random.seed(10)
bootstrap_medians = []
for i in range(1000):
```

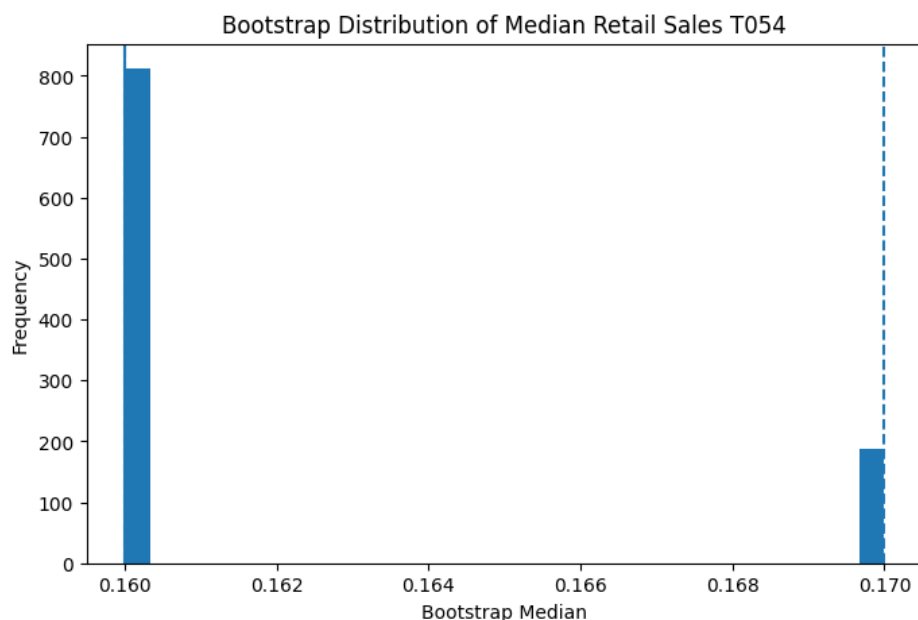
```
    sample = np.random.choice(sales, size=len(sales),
                              replace=True)
    bootstrap_medians.append(np.median(sample))
bootstrap_medians = np.array(bootstrap_medians)
median = np.median(sales)
lower = np.percentile(bootstrap_medians, 2.5)
upper = np.percentile(bootstrap_medians, 97.5)
print("Original Median:", median)
```

```
print("Number of Bootstrap Samples:", 1000)
print("95% Bootstrap Confidence Interval:")
print("Lower Limit:", lower)
print("Upper Limit:", upper)
plt.figure(figsize=(8,5))
plt.hist(bootstrap_medians, bins=30)
plt.axvline(lower, linestyle="--")
```

OUTPUT:

```
Original Median: 0.16
Number of Bootstrap Samples: 1000
95% Bootstrap Confidence Interval:
Lower Limit: 0.16
Upper Limit: 0.17
```

```
plt.axvline(upper, linestyle="--")
plt.axvline(median, linestyle="-")
plt.title("Bootstrap Distribution of Median Retail
Sales T054")
plt.xlabel("Bootstrap Median")
plt.ylabel("Frequency")
plt.show()
```



Graph:

The histogram generated by the code represents the Bootstrap Distribution of Median Retail Sales. The vertical lines indicate the original median and the lower and upper limits of the 95% confidence interval.

Interpretation:

The 95% bootstrap confidence interval gives a range of plausible values for the population median of Retail Sales based on repeated resampling from the available data.

In the business context, this helps the retail business understand the typical level of retail sales while accounting for sampling variability. The bootstrap method is useful because it estimates the uncertainty of the median without requiring the retail-sales data to follow a specific distribution.

Result: Bootstrapping was successfully performed using 1,000 samples with replacement, and a 95% confidence interval for the median Retail Sales was calculated.

19. HYPOTHESIS TESTING

Business Question

“Is the average Retail Sales significantly different from 100?”

Here, 100 is used as a benchmark value for the hypothesis test.

Hypotheses

- Null Hypothesis (H_0): The mean Retail Sales is equal to 100.
- Alternative Hypothesis (H_1): The mean Retail Sales is significantly different from 100.
- Significance Level (α): 0.05

Since we are comparing one sample mean with a fixed value, a one-sample t-test is appropriate.

CODE:

```
import pandas as pd
from scipy.stats import ttest_1samp
df = pd.read_csv("Retail and warehouse Sale.csv")
```

```
sales = df["RETAIL SALES"].dropna()
benchmark = 100
t_statistic, p_value = ttest_1samp(sales, benchmark)
```

```

print("Business Question:")
print("Is the average Retail Sales significantly
different from 100?")
print("\nNull Hypothesis (H0): Mean Retail Sales =
100")
print("Alternative Hypothesis (H1): Mean Retail Sales
!= 100")
print("Significance Level:", 0.05)
print("\nTest Statistic:", t_statistic)

```

OUTPUT:

```

Business Question:
Is the average Retail Sales significantly different from 100?

Null Hypothesis (H0): Mean Retail Sales = 100
Alternative Hypothesis (H1): Mean Retail Sales != 100
Significance Level: 0.05
Test Statistic: -487.23452428518704
p-value: 0.0
Decision: Reject H0
Conclusion: Average Retail Sales is significantly different from 100.

```

Business Interpretation:

If $p\text{-value} < 0.05$, the null hypothesis is rejected, indicating that the observed average Retail Sales is statistically different from the benchmark of 100.

If $p\text{-value} \geq 0.05$, the null hypothesis is not rejected because there is insufficient statistical evidence that the average Retail Sales differs from 100.

20. A/B TESTING

Business Scenario:

Suppose the retail website wants to test two different buttons on its product page:

- Version A: "Buy Now"
- Version B: "Shop Now"

The purpose is to determine which button results in a higher number of customer purchases.

For this example, we use:

- Version A: 1,000 visitors, 120 conversions
- Version B: 1,000 visitors, 150 conversions

1. Conversion Rate

Conversion Rate = $\text{Conversions} / \text{Visitors} \times 100$

- Version A = $120 / 1000 \times 100 = 12\%$
- Version B = $150 / 1000 \times 100 = 15\%$

Therefore, Version B has a higher observed conversion rate.

2. Hypotheses

- H_0 : There is no significant difference between the conversion rates of Version A and Version B.
- H_1 : There is a significant difference between the conversion rates of Version A and Version B.
- Significance Level: $\alpha = 0.05$

Since we are comparing two proportions, a two-proportion z-test is appropriate.

CODE:

```

import math
from scipy.stats import norm
visitors_A = 1000
conversions_A = 120
visitors_B = 1000
conversions_B = 150
rate_A = conversions_A / visitors_A
rate_B = conversions_B / visitors_B

```

```

print("p-value:", p_value)
if p_value < 0.05:
    print("Decision: Reject H0")
    print("Conclusion: Average Retail Sales is
significantly different from 100.")
else:
    print("Decision: Do not reject H0")
    print("Conclusion: There is not enough evidence to
say that Average Retail Sales is different from 100.")

```

```

pooled_rate = (conversions_A + conversions_B) /
(visitors_A + visitors_B)
standard_error = math.sqrt(
    pooled_rate * (1 - pooled_rate) *
    (1 / visitors_A + 1 / visitors_B)
)
z = (rate_A - rate_B) / standard_error
p_value = 2 * (1 - norm.cdf(abs(z)))
print("Version A Conversion Rate:", rate_A * 100,
"%")
print("Version B Conversion Rate:", rate_B * 100,
"%")

```

OUTPUT:

```

Version A Conversion Rate: 12.0 %
Version B Conversion Rate: 15.0 %

Z-statistic: -1.963049807622355
p-value: 0.04964038686536876
Significance Level: 0.05
Decision: Reject H0
There is a significant difference between the two conversion rates.

```

Suitable Chart:

CODE:

```

import matplotlib.pyplot as plt
versions = ["Buy Now", "Shop Now"]
conversion_rates = [12, 15]
plt.bar(versions, conversion_rates)

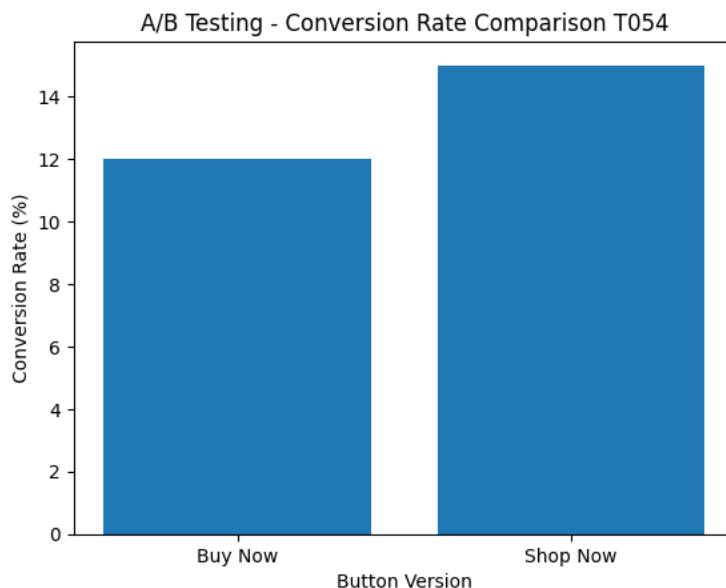
```

```

print("\nZ-statistic:", z)
print("p-value:", p_value)
print("Significance Level:", 0.05)
if p_value < 0.05:
    print("Decision: Reject H0")
    print("There is a significant difference between the
two conversion rates.")
else:
    print("Decision: Do not reject H0")
    print("There is no significant difference between
the two conversion rates.")

```

OUTPUT:



```

plt.title("A/B Testing - Conversion Rate Comparison
T054")
plt.xlabel("Button Version")
plt.ylabel("Conversion Rate (%)")
plt.show()

```

Business Interpretation:

Version B (“Shop Now”) achieved a 15% conversion rate, compared with 12% for Version A. Since the p-value is approximately 0.0496, which is less than 0.05, the null hypothesis is rejected.

Therefore, there is statistically significant evidence of a difference between the two conversion rates. Based on this test, the business can consider using Version B because it produced the higher conversion rate in this experiment.

Result: A/B testing showed that Version B performed better than Version A for the given website scenario.

21. TIME SERIES ANALYSIS

For this project, use MONTH as the time variable and RETAIL SALES as the sales variable.

Important: selected Retail and warehouse Sale.csv dataset has YEAR and MONTH, rather than a ready-made DATE column. So created a date from them. If the dataset contains only one year, the time-series analysis it will cover the months available in that year; it will not support a genuine multi-year seasonal comparison.

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.holtwinters import
ExponentialSmoothing
from statsmodels.tsa.arima.model import ARIMA
# Load dataset
df = pd.read_csv("Retail and warehouse Sale.csv")
# Create DATE column
df["DATE"] = pd.to_datetime(
    df["YEAR"].astype(str) + "-" +
    df["MONTH"].astype(str) + "-01"
)
# Calculate monthly retail sales
monthly = df.groupby("DATE")["RETAIL
SALES"].sum().sort_index()
print("Monthly Retail Sales:")
print(monthly)
print("\nNumber of Months:", len(monthly))
# Time Series Graph
plt.figure(figsize=(10,5))
plt.plot(monthly.index, monthly.values, marker="o")
plt.title("Monthly Retail Sales T054")
plt.xlabel("Month")
plt.ylabel("Retail Sales")
plt.xticks(rotation=45)
plt.grid()
plt.tight_layout()
plt.show()
# Prepare data for forecasting
sales = monthly.values
# Use simple numeric index
sales_series = pd.Series(sales)
# Exponential Smoothing
ets_model = ExponentialSmoothing(
    sales_series,
    trend="add",
    seasonal=None
).fit()
ets_forecast = ets_model.forecast(3)
print("\nExponential Smoothing Forecast:")
print(ets_forecast)
# ARIMA
arima_model = ARIMA(
    sales_series,
    order=(1,1,0)
).fit()
arima_forecast = arima_model.forecast(steps=3)
print("\nARIMA Forecast:")
print(arima_forecast)
# Future Month Dates
last_date = monthly.index[-1]
future_dates = pd.date_range(
    start=last_date + pd.DateOffset(months=1),
    periods=3,
    freq="MS"
)
# Forecast Comparison
comparison = pd.DataFrame({
    "Month": future_dates,
    "Exponential Smoothing": ets_forecast.values,
    "ARIMA": arima_forecast.values
})
print("\nForecast Comparison:")
print(comparison)
# Forecast Graph
plt.figure(figsize=(10,5))
plt.plot(
    monthly.index,
    monthly.values,
    marker="o",
    label="Actual Sales"
)
plt.plot(
    future_dates,
    ets_forecast.values,
    marker="o",
    linestyle="--",
    label="Exponential Smoothing"
)
plt.plot(
    future_dates,
    arima_forecast.values,
    marker="o",
    linestyle="--",
    label="ARIMA"
)
```

```
plt.title("Retail Sales Forecast T054")
plt.xlabel("Month")
plt.ylabel("Retail Sales")
plt.legend()
```

```
plt.grid()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

OUTPUT:

Monthly Retail Sales:

DATE

2020-01-01 74318.77

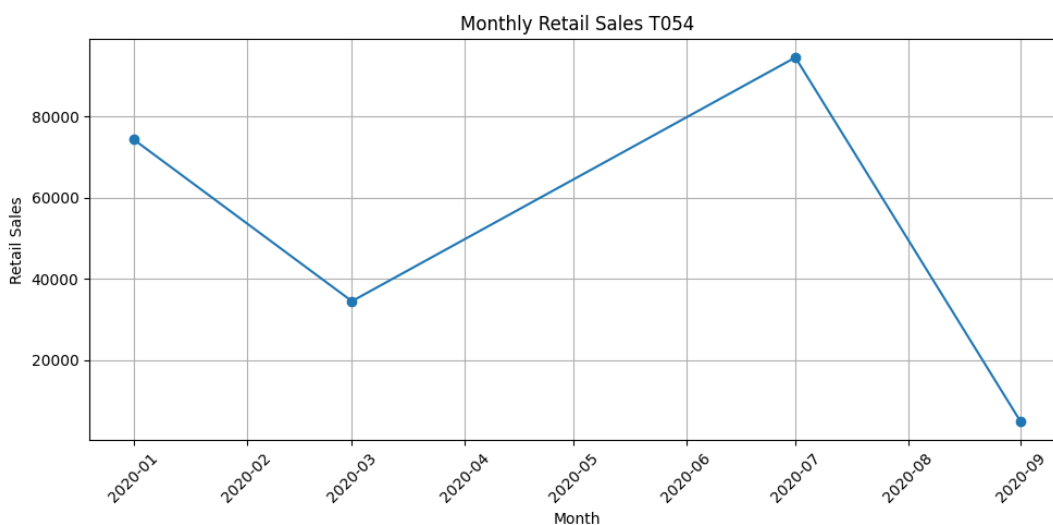
2020-03-01 34523.90

2020-07-01 94538.96

2020-09-01 4805.31

Name: RETAIL SALES, dtype: float64

Number of Months: 4



Exponential Smoothing Forecast:

4 19520.293467

5 6909.395181

6 -5701.503104

dtype: float64

ARIMA Forecast:

4 94538.959552

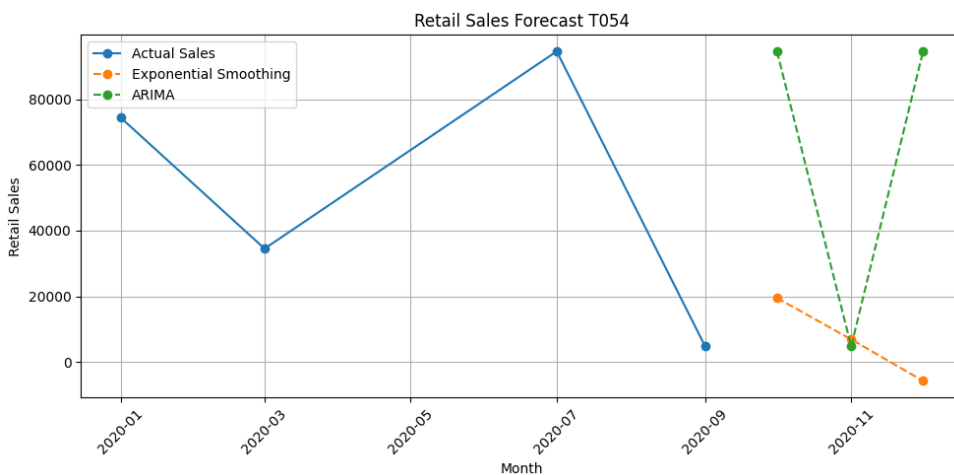
5 4805.310448

6 94538.959105

Name: predicted_mean, dtype: float64

Forecast Comparison:

Month	Exponential Smoothing	ARIMA
0 2020-10-01	19520.293467	94538.959552
1 2020-11-01	6909.395181	4805.310448
2 2020-12-01	-5701.503104	94538.959105



Result / Interpretation:

The time-series analysis shows the pattern of monthly Retail Sales over the available period. Exponential Smoothing and ARIMA are applied to generate future sales forecasts.

The two forecasting methods can be compared to see how their predicted values differ. These forecasts can support sales planning, inventory management and warehouse planning.

22. FORECAST ACCURACY

CODE:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
# Load dataset
df = pd.read_csv("Retail and warehouse Sale.csv")
# Create date
df["DATE"] = pd.to_datetime(
    df["YEAR"].astype(str) + "-" +
    df["MONTH"].astype(str) + "-01"
)
# Monthly retail sales
monthly = df.groupby("DATE")["RETAIL
SALES"].sum().sort_index()
print("Monthly Retail Sales:")
print(monthly)
print("\nNumber of Monthly Records:", len(monthly))
# Check number of observations
if len(monthly) < 3:
    print("\nNot enough monthly observations for
forecast accuracy.")
else:
    # Keep last month as test data
    train = monthly.iloc[:-1]
    test = monthly.iloc[-1:]
    print("\nTraining Data:")
    print(train)
    print("\nActual Test Data:")
    print(test)
    # Simple Forecast
    # Forecast = average of training data
    prediction = train.mean()
    predicted = np.array([prediction])
    # Calculate Errors
    actual = test.values
    mae = np.mean(np.abs(actual - predicted))
    mse = np.mean((actual - predicted) ** 2)
    rmse = np.sqrt(mse)
# Results
```

```
results = pd.DataFrame({
    "Model": ["Mean Forecast"],
    "MAE": [mae],
    "MSE": [mse],
    "RMSE": [rmse]
})
print("\nForecast Accuracy Results:")
print(results.round(2))
# Actual vs Predicted
comparison = pd.DataFrame({
    "Actual Sales": actual,
    "Predicted Sales": predicted
})
print("\nActual vs Predicted:")
print(comparison.round(2))
# Graph
plt.figure(figsize=(8,5))
plt.plot(
    monthly.index,
    monthly.values,
    marker="o",
    label="Actual Sales"
)
plt.scatter(
    test.index,
    predicted,
    label="Predicted Sales"
)

plt.title("Actual vs Predicted Retail Sales T054")
plt.xlabel("Month")
plt.ylabel("Retail Sales")
plt.legend()
plt.grid()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

OUTPUT:

Monthly Retail Sales:

DATE

2020-01-01 74318.77

2020-03-01 34523.90

2020-07-01 94538.96

2020-09-01 4805.31

Name: RETAIL SALES, dtype: float64

Number of Monthly Records: 4

Training Data:

DATE

2020-01-01 74318.77

2020-03-01 34523.90

2020-07-01 94538.96

Name: RETAIL SALES, dtype: float64

Actual Test Data:

DATE

2020-09-01 4805.31

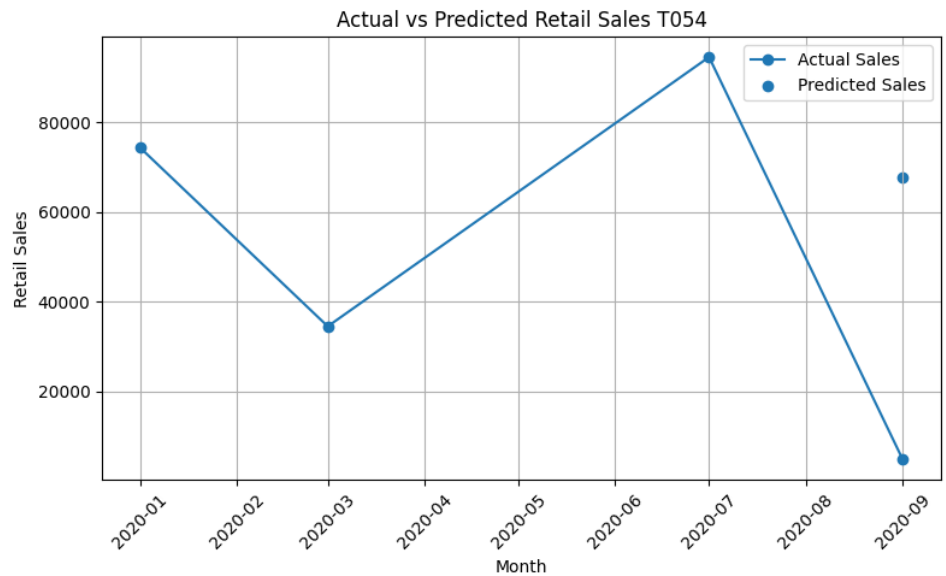
Name: RETAIL SALES, dtype: float64

Forecast Accuracy Results:

	Model	MAE	MSE	RMSE
0	Mean Forecast	62988.57	3.967560e+09	62988.57

Actual vs Predicted:

	Actual Sales	Predicted Sales
0	4805.31	67793.88



Meaning of the measures

MAE (Mean Absolute Error):

It shows the average absolute difference between actual sales and predicted sales. A lower MAE means the predictions are closer to the actual values.

MSE (Mean Squared Error):

It calculates the average squared difference between actual and predicted sales. Larger errors are given more importance because the errors are squared. A lower MSE indicates better prediction accuracy.

RMSE (Root Mean Squared Error):

It is the square root of MSE. It represents the typical prediction error in the same unit as Retail Sales. A lower RMSE indicates that the forecasts are closer to actual sales.

23. OPTIMIZATION MODELING

Business Problem:

A retail and warehouse business sells two types of products: Product A and Product B. The business has limited machine hours and labour hours. The objective is to determine how many units of each product should be produced to maximize total profit.

1. Decision Variables:

Let:

- x = number of units of Product A
- y = number of units of Product B

2. Objective Function:

Profit from:

- Product A = ₹40 per unit
- Product B = ₹30 per unit

Therefore:

- Maximize Profit: $Z=40x+30y$

CODE:

```
from scipy.optimize import linprog
# Objective function
# Negative values because linprog minimizes
profit = [-40, -30]
# Constraints
A = [
    [2, 1],
    [1, 2]
]
b = [100, 80]
# Non-negative variables
bounds = [
    (0, None),
    (0, None)
]
# Solve
result = linprog(
```

```
    profit,
    A_ub=A,
    b_ub=b,
    bounds=bounds,
    method="highs")
# Results
x = result.x[0]
y = result.x[1]
max_profit = -result.fun
machine_used = 2*x + y
labour_used = x + 2*y
print("Product A:", x)
print("Product B:", y)
print("Maximum Profit:", max_profit)
print("Machine Hours Used:", machine_used)
print("Labour Hours Used:", labour_used)
```

OUTPUT:

```
Product A: 40.0
Product B: 20.0
Maximum Profit: 2200.0
Machine Hours Used: 100.0
Labour Hours Used: 80.0
```

Business Interpretation:

The optimization model determines the production quantity that maximizes profit while satisfying the available machine and labour resources.

The optimal production plan is 40 units of Product A and 20 units of Product B, giving a maximum profit of ₹2,200.

The solution uses 100 machine hours and 80 labour hours, which means both available resources are fully utilized.

24. SIMULATION MODELING

Business Scenario: Inventory Management

A retail and warehouse business needs to decide how much inventory to keep in stock. Customer demand changes from day to day, so simulation can be used to compare different inventory levels.

In this simulation, three inventory scenarios are compared:

- **Scenario 1:** 30 units
- **Scenario 2:** 40 units
- **Scenario 3:** 50 units

Random daily demand is generated for 30 days.

Performance Measures:

For each scenario, we calculate:

- Total Sales
- Total Shortage
- Total Leftover Inventory
- Service Level

CODE:

```
import numpy as np
import pandas as pd
```

```
import matplotlib.pyplot as plt
# Random demand for 30 days
```

```

np.random.seed(10)
demand = np.random.randint(15, 51, 30)
inventory_levels = [30, 40, 50]
results = []
for inventory in inventory_levels:
    total_sales = 0
    total_shortage = 0
    total_leftover = 0
    for daily_demand in demand:
        sales = min(inventory, daily_demand)
        shortage = max(daily_demand - inventory, 0)
        leftover = max(inventory - daily_demand, 0)
        total_sales += sales
        total_shortage += shortage
        total_leftover += leftover
    service_level = total_sales / (total_sales +
total_shortage) * 100
    results.append([
        inventory,
        total_sales,
        total_shortage,
        total_leftover,
        service_level
    ])

```

OUTPUT:

Simulation Results:

	Inventory Level	Total Sales	Total Shortage	Total Leftover \
0	30	806	177	94
1	40	943	40	257
2	50	983	0	517

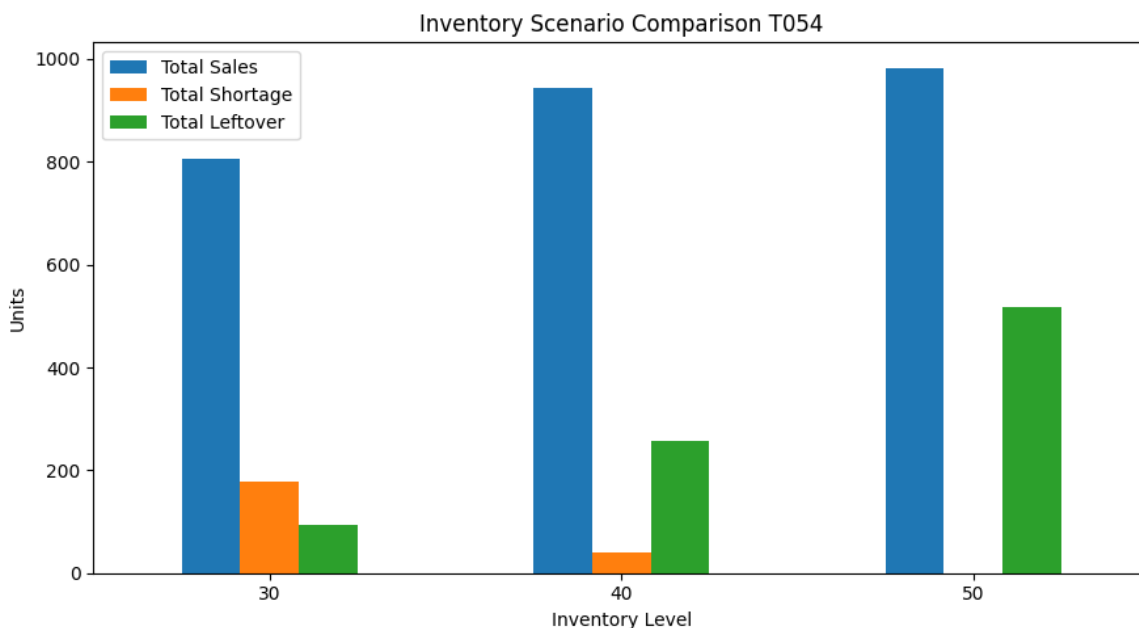
Service Level (%)

0	81.99
1	95.93
2	100.00

```

# Create results table
results_df = pd.DataFrame(
    results,
    columns=[
        "Inventory Level",
        "Total Sales",
        "Total Shortage",
        "Total Leftover",
        "Service Level (%)" ])
print("Simulation Results:")
print(results_df.round(2))
# Comparison Chart
results_df.plot(
    x="Inventory Level",
    y=["Total Sales", "Total Shortage", "Total
Leftover"],
    kind="bar",
    figsize=(9,5))
plt.title("Inventory Scenario Comparison T054")
plt.xlabel("Inventory Level")
plt.ylabel("Units")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

```



Explanation of Results:

The simulation shows how different inventory levels affect sales, shortage, leftover inventory and service level.

- A lower inventory level can result in higher shortages when customer demand is greater than available stock.
- Increasing inventory generally allows the business to satisfy more demand and therefore can increase total sales and service level.
- However, higher inventory can also result in more leftover stock when demand is lower.
- Therefore, inventory planning involves balancing product availability against excess inventory.

Business Meaning:

This simulation can help the retail and warehouse business understand the possible effects of different inventory levels under uncertain demand. It provides a simple way to compare inventory policies before making an actual business decision.

Conclusion: The simulation demonstrates that inventory decisions should consider both shortage and leftover inventory, rather than looking only at sales. The scenario results can be used to support inventory planning and warehouse management.

25. ANOVA

Business Problem

The retail business wants to determine whether **average Retail Sales are different among different Item Types**.

Since the dataset contains multiple ITEM TYPE categories, **One-Way ANOVA** can be used to compare their average Retail Sales.

Hypotheses

H_0 (Null Hypothesis):

The mean Retail Sales are equal for all Item Types.

H_1 (Alternative Hypothesis):

At least one Item Type has a different mean Retail Sales.

Significance Level: $\alpha = 0.05$

CODE:

```
import pandas as pd
from scipy.stats import f_oneway
# Load dataset
df = pd.read_csv("Retail and warehouse Sale.csv")
# Remove missing values
data = df[["ITEM TYPE", "RETAIL SALES"]].dropna()
# Display item types
print("Item Types:")
print(data["ITEM TYPE"].unique())
# Create groups
groups = []
for item_type in data["ITEM TYPE"].unique():
    group = data[data["ITEM TYPE"] ==
item_type]["RETAIL SALES"]
    groups.append(group)
```

OUTPUT:

```
Item Types:
['WINE' 'BEER' 'LIQUOR' 'STR_SUPPLIES' 'KEGS' 'REF' 'DUNNAGE'
 'NON-ALCOHOL']
```

```
One-Way ANOVA Results
F-statistic: 123.5215533834616
p-value: 8.685629095381161e-180
Significance Level: 0.05
```

```
Decision: Reject H0
Conclusion: At least one Item Type has a different mean Retail Sales.
```

```
# Perform One-Way ANOVA
f_statistic, p_value = f_oneway(*groups)
print("\nOne-Way ANOVA Results")
print("F-statistic:", f_statistic)
print("p-value:", p_value)
print("Significance Level: 0.05")
# Decision
if p_value < 0.05:
    print("\nDecision: Reject H0")
    print("Conclusion: At least one Item Type has a
different mean Retail Sales.")
else:
    print("\nDecision: Do not reject H0")
    print("Conclusion: There is not enough evidence to
say that the mean Retail Sales differ among Item
Types.")
```

26. EXPERIMENTAL DESIGN

Aim

To design an experiment to study the effect of different business factors on Retail Sales.

Experimental Design

Business Problem:

The experiment is designed to study whether different Item Types and Years have an effect on Retail Sales.

Component	Description
Factors	Item Type and Year
Levels	Different categories of Item Type and different years available in the dataset
Response Variable	Retail Sales
Experimental Units	Individual sales records/products in the dataset

Factors and Levels

1. **Factor 1 – Item Type:**

Different item categories available in the ITEM TYPE column.

2. **Factor 2 – Year:**

Different years available in the YEAR column.

3. **Response Variable – Retail Sales:**

The value recorded in the RETAIL SALES column is measured as the business outcome.

How the Experiment Was Conducted

1. The Retail and Warehouse Sales dataset was loaded using Pandas.
2. ITEM TYPE and YEAR were selected as the factors.
3. Different categories of Item Type and available years were treated as the levels of the factors.
4. RETAIL SALES was selected as the response variable.
5. The average Retail Sales for each combination of Item Type and Year was calculated.
6. The results were visualized using a bar chart to compare the groups.
7. A factorial ANOVA can then be used to study:
 - o The main effect of Item Type.
 - o The main effect of Year.
 - o The interaction effect between Item Type and Year.
8. The significance level was taken as 0.05.

CODE:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
df = pd.read_csv("Retail and warehouse Sale.csv")
data = df[["ITEM TYPE", "YEAR", "RETAIL SALES"]].dropna(
print("Factors:")
print("1. ITEM TYPE")
print("2. YEAR")
print("\nResponse Variable:")
print("RETAIL SALES")
```

```
print("\nAverage Retail Sales:")
print(data.groupby(["ITEM TYPE",
"YEAR"])["RETAIL SALES"].mean())
sns.barplot(data=data, x="ITEM TYPE", y="RETAIL SALES", hue="YEAR")
plt.title("Experimental Design - Retail Sales T054")
plt.xlabel("Item Type")
plt.ylabel("Retail Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

OUTPUT:

Factors:

1. ITEM TYPE
2. YEAR

Response Variable:
RETAIL SALES

Average Retail Sales:

ITEM TYPE	YEAR	
BEER	2020	14.118748
DUNNAGE	2020	0.000000
KEGS	2020	0.000000
LIQUOR	2020	13.635171
NON-ALCOHOL	2020	31.742419
REF	2020	5.783750
STR_SUPPLIES	2020	5.485714
WINE	2020	3.195334

Name: RETAIL SALES, dtype: float64



Conclusion

This experimental design helps study how Item Type and Year are related to Retail Sales. Comparing the different factor levels can help understand variations in sales and can support business decisions related to product planning, sales management and inventory management.

27. FACTORIAL EXPERIMENT

Aim

To study the effect of Item Type and Year on Retail Sales and determine whether the factors have individual or combined effects.

Factors and Response

Components	Description
Factor 1	ITEM TYPE
Factor 2	YEAR
Response Variable	RETAIL SALES
Experimental Units	Individual sales records

Effects to Analyze:

1. Main Effect of Item Type
Determines whether average Retail Sales differ among different item types.
2. Main Effect of Year
Determines whether average Retail Sales differ between the available years.

3. Interaction Effect (Item Type × Year)

Determines whether the effect of Item Type on Retail Sales changes depending on the Year.

Statistical Analysis:

A Two-Way Factorial ANOVA is used.

Null Hypotheses:

- H_{01} : Item Type has no significant effect on Retail Sales.
- H_{02} : Year has no significant effect on Retail Sales.
- H_{03} : There is no significant interaction between Item Type and Year.

Alternative Hypotheses:

- H_{11} : Item Type has a significant effect.
- H_{12} : Year has a significant effect.
- H_{13} : There is a significant interaction between Item Type and Year.

Significance Level: $\alpha = 0.05$

CODE:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import statsmodels.api as sm
from statsmodels.formula.api import ols

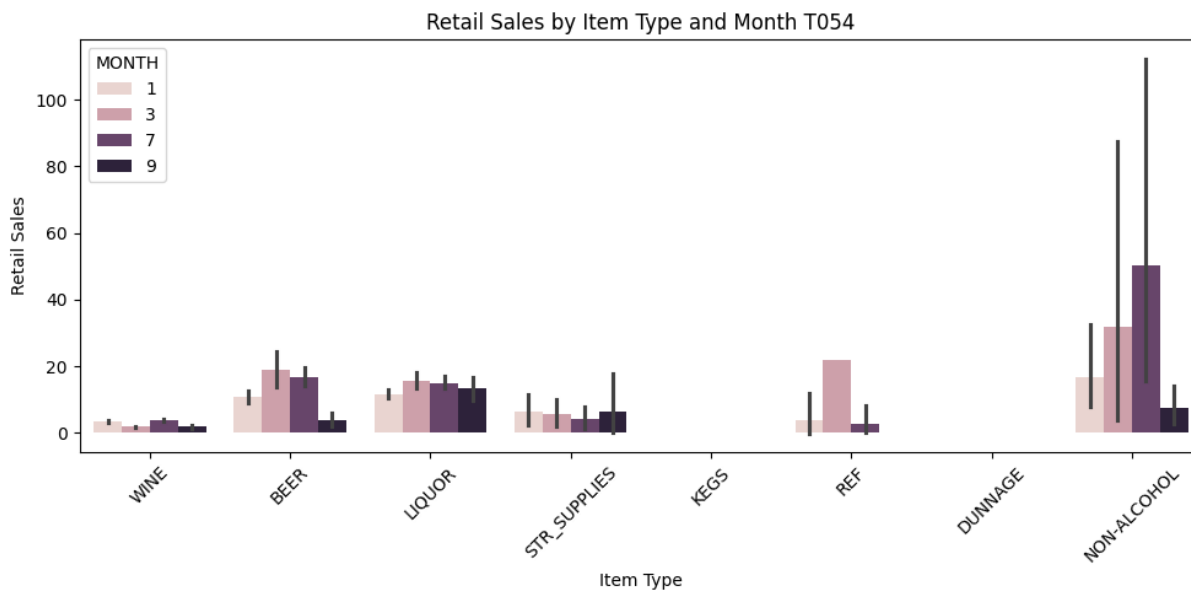
df = pd.read_csv("Retail and warehouse Sale.csv")
data = df[["ITEM TYPE", "MONTH", "RETAIL SALES"]].dropna()
print("Factors:")
print("1. ITEM TYPE")
print("2. MONTH")
print("\nResponse Variable:")
print("RETAIL SALES")
print("\nAverage Retail Sales:")
print(data.groupby(["ITEM TYPE", "MONTH"])["RETAIL SALES"].mean())
plt.figure(figsize=(10,5))
sns.barplot(
    data=data,
    x="ITEM TYPE",
    y="RETAIL SALES",
    hue="MONTH"
)
plt.title("Retail Sales by Item Type and Month T054")
plt.xlabel("Item Type")
plt.ylabel("Retail Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
model = ols(
    "Q('RETAIL SALES') ~ C(Q('ITEM TYPE')) * C(MONTH)",
    data=data
).fit()
anova = sm.stats.anova_lm(model, typ=2)
print("\nFactorial ANOVA Results:")
print(anova)
```

OUTPUT:

```
Factors:
1. ITEM TYPE
2. MONTH

Response Variable:
RETAIL SALES

Average Retail Sales:
ITEM TYPE  MONTH
BEER      1      10.723721
          3      19.030859
          7      16.719219
          9       3.812588
...
          3      1.937884
          7      3.782864
          9      1.848079
Name: RETAIL SALES, dtype: float64
```



Factorial ANOVA Results:

	sum_sq	df	F	PR(>F)
C(Q('ITEM TYPE'))	1.060519e+06	7.0	142.976043	1.890690e-179
C(MONTH)	5.009619e+04	3.0	15.758916	1.444063e-07
C(Q('ITEM TYPE')):C(MONTH)	1.258340e+05	21.0	5.654855	5.751049e-15
Residual	3.175520e+07	29968.0	NaN	NaN

Result Interpretation

The ANOVA table provides F-statistics and p-values for Item Type, Year and their interaction.

- If p-value < 0.05, the corresponding effect is statistically significant.
- If p-value $\geq$ 0.05, there is not enough evidence to consider that effect statistically significant.

Business Interpretation

The factorial experiment helps the business understand whether different item types, different years, or a combination of both are associated with differences in Retail Sales. This information can support product planning, sales analysis, inventory management and business planning.

28. BUSINESS INSIGHTS AND FINDINGS

Based on the analyses performed on Retail and warehouse Sale.csv, the following business insights can be included. Since some of the numerical outputs such as ANOVA, regression, confidence interval and forecasting values depend on your actual Python output, replace the [value] placeholders with your results rather than inventing values.

1. Retail Sales differ across Item Types

The Item Type vs Retail Sales bar chart and descriptive statistics show differences in sales among item categories. The Item Type with the highest average/total Retail Sales represents an important contributor to overall sales. Supported by: Item Type sales chart, descriptive statistics table.

2. Retail Sales and Retail Transfers have a measurable relationship

The correlation analysis between Retail Sales and Retail Transfers produced a correlation coefficient of [insert value]. This indicates the strength and direction of the relationship between product transfers and retail sales. Supported by: Correlation matrix and heatmap.

3. Retail Transfers can be used to estimate Retail Sales

The regression model using Retail Transfers as the independent variable and Retail Sales as the dependent variable produced an R^2 value of [insert value]. The regression coefficient was [insert value], showing how predicted Retail Sales changes with Retail Transfers.

Supported by: Regression model, R^2 , regression equation and scatter plot.

4. Retail Sales show variation in their distribution

The histogram and box plot of Retail Sales show the distribution and spread of sales values. The calculated standard deviation was [insert value], while the skewness was [insert value].

This indicates the degree of variation in sales and helps identify unusually high or low sales observations.

Supported by: Histogram, box plot and descriptive statistics.

5. Average Retail Sales can be estimated using confidence intervals

The calculated 95% confidence interval for the mean Retail Sales was:

[Lower Limit] to [Upper Limit]

This provides an estimated range for the population mean Retail Sales and can support sales planning and business forecasting.

Supported by: Confidence interval table.

6. The median Retail Sales has measurable uncertainty

Bootstrap analysis using 1,000 resamples produced a 95% bootstrap confidence interval of [Lower Limit] to [Upper Limit] for the median Retail Sales.

This shows the range of plausible values for the population median based on repeated resampling.

Supported by: Bootstrap distribution and confidence interval table.

7. Item Type has a statistically significant/non-significant effect on Retail Sales

The one-way ANOVA comparing Retail Sales across Item Types produced an F-statistic of [insert value] and p-value of [insert value].

Since the p-value is [less than/equal to/greater than] 0.05, the null hypothesis was [rejected/not rejected].

Therefore, the analysis provides [evidence/no sufficient evidence] that average Retail Sales differ among Item Types.

Supported by: ANOVA table and Item Type box plot.

8. Monthly sales patterns provide information for business planning

The monthly Retail Sales analysis shows how sales vary across the available months. The month with the highest recorded Retail Sales was [insert month], while [insert month] recorded the lowest.

This information can help the business plan inventory and sales activities according to observed monthly patterns.

Supported by: Monthly Retail Sales line chart and time-series table.

9. Item Type and Month can be studied together using factorial analysis

A factorial experiment was designed using ITEM TYPE and MONTH as factors and RETAIL SALES as the response variable. The analysis examines the main effects of both factors and their interaction.

The factorial ANOVA results showed:

- Item Type p-value = [insert value]
- Month p-value = [insert value]
- Item Type $\times$ Month p-value = [insert value]

These results help identify whether sales differences are related to product category, month, or their combined effect.

Supported by: Factorial ANOVA table and Item Type–Month comparison chart.

29. BUSINESS RECOMMENDATIONS

Based on the analysis of Retail Sales, Retail Transfers, Warehouse Sales, Item Type and monthly sales patterns, the following recommendations can be made:

1. Focus on high-performing Item Types

Identify Item Types with higher average or total Retail Sales and give them greater attention in product planning and inventory allocation.

2. Use Retail Transfers for sales planning

Since Retail Transfers were analyzed in relation to Retail Sales, the business can monitor transfer activity to support better sales estimation and product movement decisions.

3. **Maintain sufficient inventory during high-sales months**

The monthly sales analysis should be used to identify periods with higher Retail Sales and ensure sufficient stock is available during those periods.

4. **Use forecasting for future inventory planning**

Exponential Smoothing and ARIMA forecasts can be used as supporting information for estimating future Retail Sales and planning inventory requirements.

5. **Monitor low-performing Item Types**

Item Types with consistently lower sales should be reviewed to understand demand, product performance and inventory requirements before allocating additional stock.

6. **Use statistical analysis for product decisions**

ANOVA results should be considered when comparing sales across Item Types. If significant differences are found, inventory and sales strategies can be adjusted according to the observed differences.

7. **Monitor Item Type and Month together**

The factorial analysis of Item Type and Month can help identify combinations associated with different sales levels. This can support month-wise product and inventory planning.

8. **Use dashboards for regular monitoring**

The Business Intelligence dashboard should be updated and used to monitor Retail Sales, Warehouse Sales, Retail Transfers, Item Type performance and monthly trends, helping management make data-based decisions.

30. CONCLUSION

This project focused on analyzing retail sales and warehouse operations using the dataset Retail and warehouse Sale.csv. The main problem studied was how business data related to Item Type, Suppliers, Retail Sales, Retail Transfers and Warehouse Sales can be analyzed to understand sales performance and support better business decisions.

Various Business Analytics and statistical techniques were applied in the project. These included data preprocessing, exploratory data analysis, descriptive statistics, distribution analysis, correlation, regression, Business Intelligence dashboards, probability, decision-making under uncertainty, sampling distributions, confidence intervals, bootstrapping, hypothesis testing, A/B testing, time-series analysis, forecasting, optimization, simulation, ANOVA and factorial experimental design.

The analysis helped identify differences in sales across Item Types, relationships between retail sales and transfers, variation in sales values, monthly sales patterns and possible future sales estimates. Statistical methods such as regression, ANOVA and hypothesis testing provided quantitative evidence for evaluating business relationships and differences.

The findings can help the business improve sales planning, inventory management, product planning, warehouse operations and future forecasting. The Business Intelligence dashboard can also be used to monitor important sales indicators and identify changes in performance.

Through this project, it was learned how raw business data can be converted into meaningful information using statistical analysis, visualization and analytical models. The project also provided practical understanding of how Business Analytics techniques can be applied to real business problems and used to support data-driven decision-making.

31. APPENDIX

The Appendix contains the supporting material used during the project. It provides the code, datasets, outputs, charts and other evidence used for the analysis.

A. Complete Python Code

Include the complete Python programs used for:

- Data loading and preprocessing
- Exploratory Data Analysis
- Descriptive statistics
- Distribution analysis
- Correlation analysis
- Regression analysis
- Probability and distributions
- Sampling and sampling distributions
- Confidence interval estimation
- Bootstrapping
- Hypothesis testing
- A/B testing
- Time-series analysis and forecasting
- Optimization
- Simulation
- ANOVA
- Factorial experiment

B. Dataset Sample

Include a screenshot of the **first 5–10 rows** of Retail and warehouse Sale.csv showing columns such as:

- YEAR
- MONTH
- SUPPLIER
- ITEM CODE
- ITEM DESCRIPTION
- ITEM TYPE
- RETAIL SALES
- RETAIL TRANSFERS
- WAREHOUSE SALES

C. Important Outputs

Include screenshots of important Python outputs such as:

- df.shape
- df.head()
- Missing-value results
- Duplicate-value results
- Descriptive statistics
- Correlation matrix
- Regression results
- Confidence interval results
- Bootstrap results
- Hypothesis test results
- ANOVA results
- Forecast results
- Forecast accuracy results

D. Additional Graphs

Include the graphs generated during the project:

1. Retail Sales by Item Type
2. Top Suppliers by Retail Sales
3. Retail Sales Histogram
4. Retail Sales Box Plot
5. Correlation Heatmap
6. Retail Transfers vs Retail Sales Scatter Plot
7. Probability Distribution Graphs
8. Monthly Retail Sales Line Chart
9. Forecast Comparison Chart
10. ANOVA/Factorial Experiment Chart
11. Simulation Comparison Chart
- 12.

E. Tableau / Power BI Screenshots

Include screenshots of the Business Intelligence dashboard showing:

- Total Retail Sales
- Total Warehouse Sales
- Total Retail Transfers
- Retail Sales by Item Type
- Monthly Retail Sales
- Supplier analysis
- Filters used in the dashboard

F. Decision Tree

Include the decision tree used for the **inventory decision-making scenario**, showing:

- Decision alternatives
- High-demand and low-demand outcomes
- Probabilities
- Payoffs
- Expected values

G. Optimization Output

Include the Python output for the optimization model showing:

- Product A quantity
- Product B quantity

- Maximum profit
- Machine hours used

- Labour hours used

H. Simulation Output

Include the simulation results comparing different inventory levels and showing:

- Total Sales
- Total Shortage
- Total Leftover
- Service Level

I. Statistical Analysis Tables

Include the final tables for:

- Descriptive statistics
- Correlation
- Regression
- Confidence intervals
- Bootstrap confidence interval
- Hypothesis testing
- A/B testing
- ANOVA
- Factorial ANOVA
- Forecast accuracy